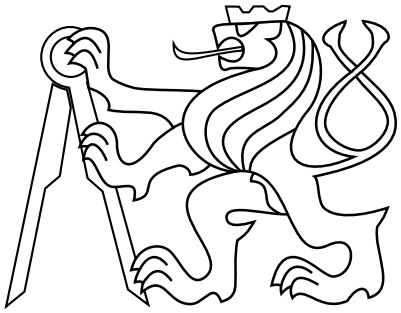
Czech Technical University in Prague

Faculty of Electrical Engineering
Department of Cybernetics
Multi-robot Systems



Rigorous Evaluation of Large-scale Mapping and Localization Systems on High-fidelity Synthetic Datasets

Bachelor's Thesis

Petr Kučera

Prague, April 2025

Study programme: Cybernetics and Robotics

Supervisor: Ing. David Čapek

Acknowledgments

First and foremost, I would like to express my gratitude to my supervisor, Ing. David Čapek, for his support and assistance during the writing of this thesis. I would also like to thank my colleagues at my workplace for their support and understanding during the busy periods of my studies. Finally, I would like to thank my family and close friends for their encouragement and emotional support. Without them, I would not be where I am today.

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Kučera** Jméno: **Petr** Osobní číslo: **524039**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra kybernetiky**

Studijní program: **Kybernetika a robotika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Rigorózní vyhodnocení systémů pro rozsáhlé mapování a lokalizaci na vysoce věrných syntetických datasetech

Název bakalářské práce anglicky:

Rigorous Evaluation of Large-scale Mapping and Localization Systems on High-fidelity Synthetic Datasets

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. David Čapek Multirobotické systémy FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Ing. Tomáš Báča, Ph.D. Multirobotické systémy FEL

Datum zadání bakalářské práce: **05.02.2026** Termín odevzdání bakalářské práce: _____

Platnost zadání bakalářské práce: **19.09.2027**

podpis vedoucí(ho) ústavu/katedry

podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Kučera Petr

Podpis studenta

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Kučera** Jméno: **Petr** Osobní číslo: **524039**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra kybernetiky**

Studijní program: **Kybernetika a robotika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Rigorózní vyhodnocení systémů pro rozsáhlé mapování a lokalizaci na vysoce věrných syntetických datasetech

Název bakalářské práce anglicky:

Rigorous Evaluation of Large-scale Mapping and Localization Systems on High-fidelity Synthetic Datasets

Pokyny pro vypracování:

Autonomous robotic systems have seen significant development in recent years. Showing a shift from fine-tuned systems that were domain-specific to general systems. This led to a need for large amounts of training and testing data. The easiest way to obtain labeled data is to use paternalistic simulators. Creating novel environments for the simulators can be time-consuming. That is what this thesis will focus on: creating an automated method for generating high-fidelity environments for the verification of existing mapping and localization systems.
The thesis will be divided into the following parts:
- Familiarize yourself with the FlightForge simulator [1] and Robot Operating System 2 (ROS2).
- Research methods for creating photorealistic environments [2, 3].
- Implement or adapt the selected method into the FlightForge simulator.
- Generate datasets containing LiDARs, RGB cameras, Ground truth segmentations for both modalities using the created environments that will be used for the verification of existing mapping and localization methods [4].
- Verify the existing localization and mapping frameworks on the generated datasets.
- Prepare the dataset for public release.

Seznam doporučené literatury:

[1]. D. Čapek, et al. “FlightForge: Advancing UAV Research with Procedural Generation of High-Fidelity Simulation and Integrated Autonomy”, 2025 IEEE International Conference on Robotics & Automation (ICRA)
[2]. Fan-Yun Sun, et al. “3D-Generalist: Self-Improving Vision-Language-Action Models for Crafting 3D Worlds”, 2026 International Conference on 3D Vision
[3]. Yan Zhuang, et al. “SimWorld-Robotics: Synthesizing Photorealistic and Dynamic Urban Environments for Multimodal Robot Navigation and Collaboration”, NeurIPS 2025
[4]. K. Ebadi et al., "Present and Future of SLAM in Extreme Environments: The DARPA SubT Challenge," in IEEE Transactions on Robotics

FAKULTA ELEKTROTECHNICKÁ
FACULTY OF ELECTRICAL ENGINEERING
Technická 2
166 27 Praha 6

DECLARATION

I, the undersigned

Student's surname, given name(s):	Kučera Petr
Personal number:	524039
Programme name:	Cybernetics and Robotics

hereby declare that the Bachelor's Thesis titled

Rigorous Evaluation of Large-scale Mapping and Localization Systems on High-fidelity Synthetic Datasets

is my own independent work and that I have declared all sources of information used in accordance with the Methodological Guideline for adhering to ethical principles when elaborating an academic final thesis and the Framework Rules for the use of artificial intelligence at CTU for study and pedagogical purposes in Bachelor and continuing Masters studies.

I declare that I have used artificial intelligence tools during the preparation and writing of the final thesis.
I've verified the generated content. I confirm that I am aware that I am fully responsible for the content of the final thesis.

In Prague on 20.05.2026

Petr Kučera
.....
student's signature

Abstract

Virtual environments are increasingly used in simulation, testing, and training scenarios. However, manual creation of such environments is a time-consuming process, especially when the environments are large outdoor worlds. This thesis focuses on a semi-automatic method for creating outdoor procedural environments while preserving user control over the generation process.

The proposed method uses height maps and segmentation images as the basis for environment generation inside Unreal Engine 5. A height map is used to create the landscape, while segmentation images are processed into binary masks. From these masks, skeletons and contours are extracted. Skeletons are used to generate roads and rivers, while contours are used to generate lakes and biome regions. Biome generation is implemented using the Unreal Engine 5 PCG Biome Core plugin.

Four worlds were generated using the proposed method. These worlds were then used to create datasets with the Flight Forge simulator and on them selected mapping frameworks were evaluated. The created datasets can be used for testing localization, mapping, LiDAR odometry and visual odometry. Additionally, if the generated environments are based on satellite imagery, the resulting datasets can also be compared with corresponding real-world locations.

Keywords UnmannedAAerial Vehicles, Flight Forge Simulator, Procedural Environment Generation, Unreal Engine 5, Segmentation, Binary Mask Processing, Spline Generation, Mapping Validation

Abstrakt

Virtuální prostředí jsou stále častěji využívána pro simulace, testování a trénovací scénáře. Ruční tvorba takových prostředí je však časově náročný proces, zejména pokud jsou tato prostředí velké venkovní světy. Tato práce se zaměřuje na poloautomatickou metodu vytváření procedurálních venkovních prostředí při zachování uživatelské kontroly nad generačním procesem.

Navržená metoda využívá výškové mapy a segmentační obrázky jako základ pro generování prostředí v Unreal Engine 5. Výšková mapa je použita k vytvoření krajiny a segmentační obrázky jsou zpracovány do binárních masek. Z těchto masek jsou extrahovány kostry a kontury. Kostry jsou použity pro generování cest a řek, zatímco kontury jsou použity pro generování jezer a oblastí biomů. Generace biomů je implementována pomocí pluginu Unreal Engine 5 PCG Biome Core.

Čtyři světy byly vygenerovány pomocí navžené meotdy. Tyto světy byly následně použity k vytvoření datasetů pomocí simulátoru Flight Forge a na těchto datasetech byly vyhodnoceny vybrané mapovací frameworky. Vytvořené datasety lze použít pro testování lokalizace, mapování, LiDARové odometrie a vizuální odometrie. Pokud jsou navíc generovaná prostředí založena na satelitních snímcích, výsledné datasety lze porovnat s odpovídajícími lokacemi v reálném světě.

Klíčová slova Bezpilotní prostředky, Flight Forge simulátor, Procedurální generace prostředí, Unreal Engine 5, Segmentace, Zpracování binárních masek, Generace křivek, Validace mapování

Abbreviations

API Application Programming Interface

LiDAR Light Detection and Ranging

ROS2 Robot Operating System 2

UAV Unmanned Aerial Vehicle

DEM Digital Evaluation Model

LLM Large Language Model

VLM Vision–Language Model

GAN Generative Adversarial Network

CGAN Conditional Generative Adversarial Networks

BEV Bird's-Eye View

EDT Euclidean Distance Transform

SDF Signed Distance Field

PCG Procedural Content Generation Framework

ESDF Euclidean Signed Distance Field

Contents

List of Figures

Chapter 1

Introduction

Recent works have shown that full autonomy of Unmanned Aerial Vehicle (UAV) remains an unsolved problem [9], while opportunities to evaluate autonomous systems in challenging real-world environments are limited. For this reason, high-fidelity virtual environments have become an essential tool for the development, testing, and validation of autonomous methods. In the context of UAVs, simulated worlds make it possible to assess navigation, perception, and decision-making algorithms in controlled conditions before deployment in real scenarios.

For such simulations to be useful, the virtual environment must contain meaningful spatial structure and sufficient visual diversity. It should allow the system to interact with terrain, obstacles, and scene elements in a way that resembles real deployment conditions. In this work, we aim to present a method for generation of visually diverse environments with minimal user input.

Virtual environments can be broadly categorized into indoor and outdoor. Indoor environments often require detailed placement of objects, furniture, and other props in order to appear believable. This can be difficult to achieve using only simple procedural rules, since the resulting scene must respect spatial relationships and common patterns of human designed spaces. Outdoor environments, on the other hand, are often more suitable for procedural or semi-automatic generation, as their structure can frequently be described using terrain elevation, paths, water bodies, vegetation regions, urban areas, and other large-scale features.

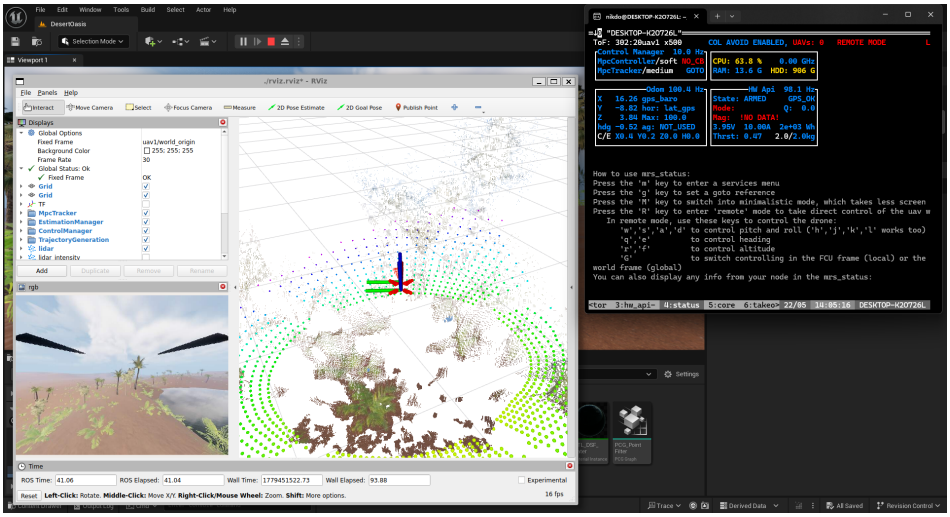


Figure 1.1: Example of the usage of Flight Forge simulator [8] on a generated world.

This thesis focuses specifically on the generation of outdoor virtual environments. Creating such environments manually is a time-consuming process. Elements such as terrain, roads, rivers, lakes, and biomes often have to be placed consistently with the intended layout

of the scene. When created manually, this process requires a significant amount of repetitive work and can become difficult to manage for larger environments.

One way to reduce this workload is to use input data that already describes the desired structure of the environment. Height maps can define the terrain elevation, while segmentation images can describe the placement of different environmental features such as roads, rivers, lakes, forests, or biomes. However, these image-based representations cannot be used directly as editable objects inside a 3D engine. They first need to be processed and converted into engine-specific structures.

This thesis focuses on the design and implementation of an Unreal Engine 5 plugin that automates part of this process. The plugin uses height maps and segmentation images to generate the initial structure of a virtual environment. Opposed to autonomous procedural generation this work focuses to giving the user of this plugin the ability to adjust any step of the generation pipeline.

By converting image-based input data into editable Unreal Engine objects, the proposed plugin reduces the amount of manual work required to build the simulation environment while preserving user control over the final result. Such environments can then be used as a basis for simulation scenarios or generation of datasets for evaluating mapping and localization methods.

The implemented plugin is designed with the Flight Forge [8] simulation workflow in mind. Its purpose is to support the creation of high-fidelity environments that can later be used for robotic simulation and dataset generation. Datasets created from environments generated by the proposed method were then used to evaluate mapping frameworks to showcase their performance on large outdoor environments.

1.1 Related Works

Many works specialize in generating environments completely procedurally to ensure their uniqueness, while other focus on creating environments based on user input. This input can be publicly available data, such as satellite imagery, manually created landscape to populate with features, or adjustable parameters defining how the procedural generation will proceed.

Recent advances in development of artificial intelligence have shown promising results regarding generation of unique environments suitable for robotic simulations. This ranges from generating the foundational data, such as height maps, with trained neural networks or creating the entire environment from user written prompt.

1.1.1 User Guided Environment Creation

A pipeline for manual creation of outdoor environments in Unreal Engine 5 was proposed in [6]. This approach uses a manually created landscape together with a hand-drawn RGB biome mask as the basis for environment generation. While this gives the user a high degree of control over the final result, it still requires significant manual effort. In addition, the pipeline focuses primarily on biome generation and does not directly address the creation of roads, rivers, lakes, or urban structures, which limits the variety of resulting simulation datasets.

A method for procedural generation of multi-story buildings with interiors was proposed in [16]. The proposed pipeline takes user defined input such as shape of the building, room and

floor count and generates three dimensional model of said building. However the algorithm does not take into account placement of furniture and other assets.

Another user guided based method was proposed in [23]. This approach requires only a user-defined building shape and room definitions. The method first generate two-dimensional floor plan before extending it into three dimensions and adding doorways and windows. A limitation mentioned by the authors an inability to create corridors, unlike the previously discussed method.

1.1.2 Generation from Real-World Data

A more automated process of generating realistic environment using satellite imagery was proposed in [4]. In this work, satellite data are converted into height map and color mask are used in automated landscape generation with biomes. However, the quality of the generated environment depends heavily on the accuracy and availability of the input satellite data.

Another procedural generation approach based on satellite imagery was presented in [20]. The proposed method provides a number of input parameters to allow broader control over the generation process, one of which is a satellite image. The satellite image is classified and used to create a more detailed and realistic terrain representation, for example by adding foliage.

1.1.3 Procedural Synthetic Environment Generation

A complete procedural terrain generation method was proposed in [17]. The method uses a Generative Adversarial Network (GAN) to procedurally generate satellite images. The generated satellite image, or alternatively an existing satellite image as in the real-world-data-based approaches discussed above, is then used as input to a Conditional Generative Adversarial Networks (CGAN) to generate the corresponding Digital Evaluation Model (DEM). These outputs can be used to define both the shape and the texture of the final terrain. However, the proposed method focuses only on terrain generation and does not populate the environment with elements such as foliage, roads, rivers, or buildings.

Procedural generation has also been used for the creation of urban simulation environments. The methods proposed in [1], [19] focus on generating synthetic urban worlds for training navigation and perception models. Proposed pipelines typically consist of road network generation, building placement and street element generation. These approaches are well suited for urban scenarios, but they are less focused on generating natural outdoor environments.

Another notable synthetic dataset generator is Infinigen [14], which is capable of generating highly diverse natural environments. Infinigen focuses on procedural generation of photorealistic scenes and is designed primarily for synthetic image dataset generation. Since it is built around Blender-based scene generation, its output is more suitable for rendering and dataset creation than for direct use as an editable Unreal Engine simulation environment. This makes it different from approaches that aim to create interactive worlds inside a real-time engine.

The original Infinigen system is focused mainly on natural scenes. To extend this idea to indoor environments, Infinigen Indoors was introduced in [11]. This system follows a similar

procedural philosophy, but targets indoor scene generation. It is capable of creating rooms, furniture arrangements, and indoor objects procedurally, reducing the need for a large manually prepared asset library.

On Infinigen Indoors generator system, another creation tool was build, Infinigen-Articulated [5]. This generator has the ability to generate simulation-ready assets, including moving parts and dynamic and kinematic definitions. This allows the user of this tool to more easily create robot simulation and testing environments without the need of creating the assets manually, which is time consuming task.

A different method for generating realistic environments was proposed in [13]. The method generates a bird's-eye-view (Bird's-Eye View (BEV)) scene representation from simplex noise, including a height field describing terrain elevation and a semantic field describing scene semantics. From this 3D representation, realistic 2D images are produced using neural volumetric rendering. The model is trained only on collections of real 2D images, which are used to learn the appearance of generated 3D worlds, without requiring explicit 3D annotations.

1.1.4 Prompt-Based Generation

Recent advances in large language models and vision-language models have introduced a different approach to virtual environment generation. Instead of requiring the user to manually define all generation parameters, these methods allow the desired scene to be described using a high-level text prompt. This makes the generation process more accessible to users, but it also reduces direct control over the exact spatial structure of the resulting environment.

Methods for generation of 3D outdoor environments with features such as biomes, roads and rivers were proposed in [3], [10]. These methods follow previously mentioned pipelines that consist of creating landscape, populating it with biomes and vegetation, objects and paths. Parameters required for executing these pipelines are converted from user input prompt via Large Language Model (LLM) processing layer.

A method for urban environment generation that utilizes an LLM for placement and control of PCG was proposed in [12]. The scene elements are categorized into roads, buildings, greenery, traffic flow, pedestrians, and water bodies. The method uses a multi-agent framework to divide the workflow into annotation, planning, execution, and evaluation stages. This workflow enables the generation of high-quality large-scale urban scenes from user descriptions as well as from real-world data, such as satellite images.

Method for generating visually coherent 3D indoor environments was proposed in [7]. It combines a LLM for prompt interpretation with a Vision–Language Model (VLM) for refining resulting environment. A limitation of this method is that the assets used to populate the interiors still have to be supplied.

1.2 Mathematical Notation

$\mathbf{x}$, $\boldsymbol{\alpha}$	vector, pseudo-vector, or tuple
$\hat{\mathbf{x}}$, $\hat{\boldsymbol{\omega}}$	unit vector or unit pseudo-vector
$\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2, \hat{\mathbf{e}}_3$	elements of the *standard basis*
$\mathbf{X}, \boldsymbol{\Omega}$	matrix
$\mathbf{I}$	identity matrix
$x = \mathbf{a}^\intercal \mathbf{b}$	inner product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \times \mathbf{b}$	cross product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \circ \mathbf{b}$	element-wise product of $\mathbf{a}$, $\mathbf{b} \in \mathbb{R}^3$
$\mathbf{x}_{(n)} = \mathbf{x}^\intercal \hat{\mathbf{e}}_n$	n^{th} vector element (row), $\mathbf{x}, \mathbf{e} \in \mathbb{R}^3$
$\mathbf{X}_{(a,b)}$	matrix element, (row, column)
x_d	x_d is *desired*, a reference
$\dot{x}, \ddot{x}, \dddot{x}, \ddddot{x}$	1^{st}, 2^{nd}, 3^{rd}, and 4^{th} time derivative of x
$x_{[n]}$	x at the sample n
$\mathbf{A}, \mathbf{B}, \mathbf{x}$	LTI system matrix, input matrix and input vector
SO(3)	3D special orthogonal group of rotations
SE(3)	*SO(3)* $\times \mathbb{R}^3$, special Euclidean group

Table 1.1: Mathematical notation, nomenclature and notable symbols.

Chapter 2

Methodology

2.1 Binary Mask Processing

2.1.1 Binary Mask Creation

A segmentation image is converted into a binary mask by selecting all pixels whose color lies within a specified range. This range can be adjusted by the user to obtain finer control over binary mask creation.

The result is a binary image in which white pixels, referred to as foreground, are assigned the value true, while black pixels, referred to as background, are assigned the value false. We refer to clusters of white or black pixels as regions.

2.1.2 Euclidean Distance Transform

EDT assigns to each pixel the Euclidean distance to the nearest pixel of a specified set. This can be used for extracting widths from binary masks.

In our implementation, the cost function f is chosen so that distances are computed from foreground pixels to the nearest background pixel. This is determined by which pixels are assigned the values 0 and ∞ in the cost function.

This process can be generalized to transform a cost function on a grid using spatial information [22], such as using distance function other than Euclidean, or n-dimensional grids.

We define cost function as $f(q)$:

$$f(q) = \begin{cases} \infty & \text{if } q \text{ is a foreground pixel,} \\ 0 & \text{otherwise,} \end{cases} \tag{2.1}$$

where q is a pixel of the binary mask. The distance transform of f is denoted by $\mathcal{D}_f$ and defined as

$$\mathcal{D}_f(p) = \min_{q \in \mathcal{G}} \left(d(p,q) + f(q) \right) \quad , \tag{2.2}$$

where $\mathcal{G}$ is the grid representing the binary mask, $p \in \mathcal{G}$, and $d(p,q)$ is a distance function.If $d(p,q)$ is chosen as Euclidean distance, then $\mathcal{D}_f$ is called the Euclidean distance transform.

Applying EDT to the binary mask yields an unsigned distance field, show on figure Fig. 2.1. The value at each element of the distance field is the distance from the corresponding pixel in the binary mask to the nearest background pixel. This can be seen in definition of function $f(q)$, where its value for black pixels is zero, meaning $\mathcal{G}$ computes distance of a white pixel to a black pixel. Consequently, the value of every background pixel in this field is 0.

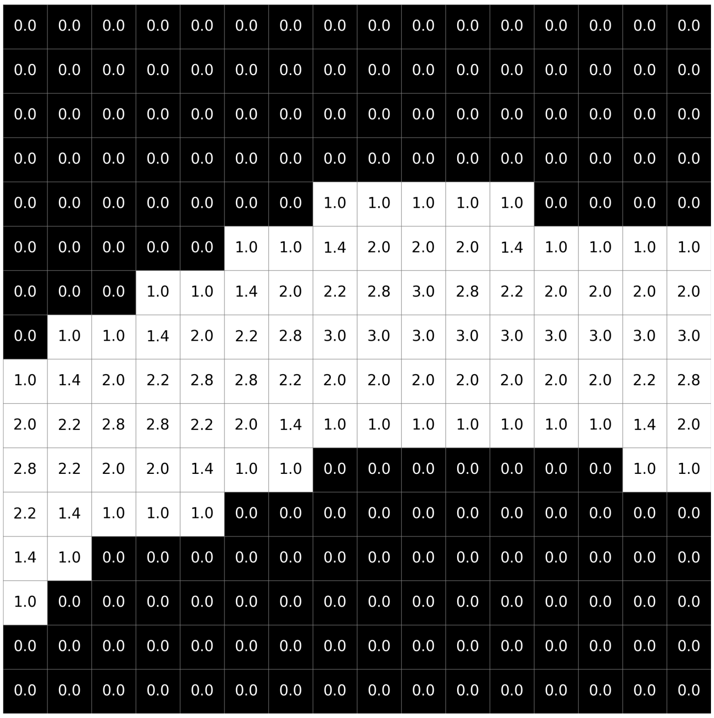


Figure 2.1: Visualization of unsigned distance field of a binary mask.

In our application, this algorithm is used to extract the widths of roads or rivers by computing EDT of foreground pixels. These widths can then be converted to approximate real-world measurements. The precision of this approach depends on the accuracy and resolution of the binary mask.

2.1.3 Signed Distance Field

Signed Distance Field (SDF), is a field in which the value corresponding to each pixel of binary mask represents the distance from that pixel to the nearest boundary between foreground and background region. Unlike the unsigned distance field, the SDF stores meaningful values on both sides of the foreground boundary. To differentiate between the two colors, we denote the distances of background pixels as negative.

First we define distance transform $\mathcal{D}_g$ for background pixels by inverting the cost function, so we compute distances of black pixel to a white pixel. The cost function $g(q)$ is

$$g(q) = \begin{cases} \infty & \text{if } q \text{ is a background pixel,} \\ 0 & \text{otherwise,} \end{cases} \tag{2.3}$$

where q is a pixel of the binary mask. The distance transform of g is denoted by $\mathcal{D}_g$ and defined as

$$\mathcal{D}_g(p) = -\min_{q \in \mathcal{G}} \left(d(p, q) + g(q) \right) \quad , \tag{2.4}$$

where $\mathcal{G}$ is the grid representing the binary mask, $p \in \mathcal{G}$, and $d(p, q)$ is the Euclidean distance.

Signed distance field, shown on Fig. 2.2, has the same dimensions as the initial binary mask. The SDF value v at pixel p is defined as the sum of these two distance transforms

$$v = \mathcal{D}_f(p) + \mathcal{D}_g(p), \tag{2.5}$$

where p is a pixel on binary mask. When p is a foreground pixel, $\mathcal{D}_f(p)$ is positive, while $\mathcal{D}_g(p) = 0$. Conversely, when p is a background pixel, $\mathcal{D}_f(p) = 0$ and $\mathcal{D}_g(p)$ in a negative value.

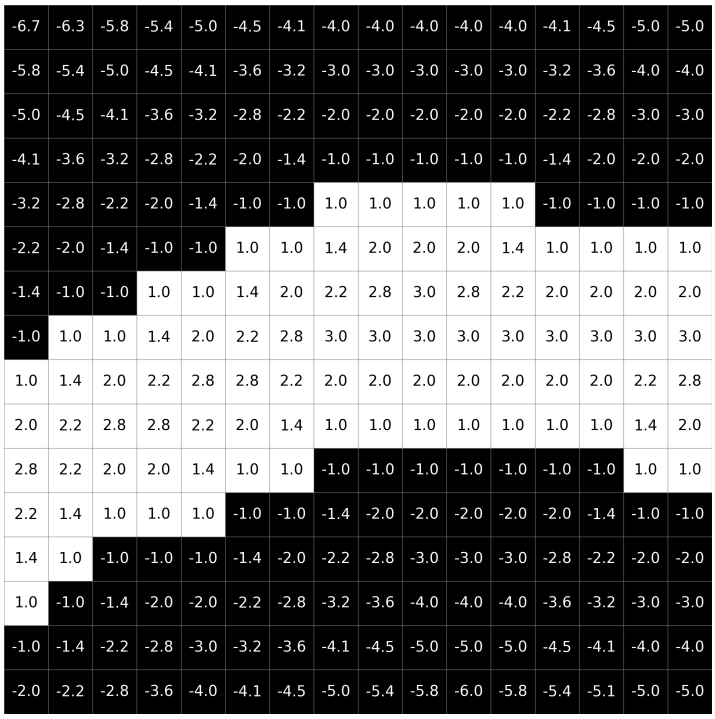


Figure 2.2: Visualization of signed distance field of a binary mask.

2.1.4 Binary Mask Smoothing

Binary mask smoothing is achieved by applying Gaussian blur to the computed signed distance field. The blurred field is then classified by a threshold. Values above a this threshold are classified as foreground, while the remaining values are classified as background. When applied to very thin foreground regions, this process may disrupt their continuity.

Gaussian function for two dimensions is defined as

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad , \tag{2.6}$$

where parameter σ determines the blur strength. Increasing σ results in stronger smoothing, while decreasing produces weaker smoothing.

To blur an image $I(x, y)$ we compute its convolution with Gaussian function

$$I_{\text{blurred}}(x, y) = (I * G)(x, y) = \sum_u \sum_v I(x - u, y - v)G(u, v). \tag{2.7}$$

In discrete implementation we use kernel of size $(2R + 1) \times (2R + 1)$, where R is a radius generally chosen as

$$R = \max\left(1, \lceil 3\sigma \rceil\right) \quad , \tag{2.8}$$

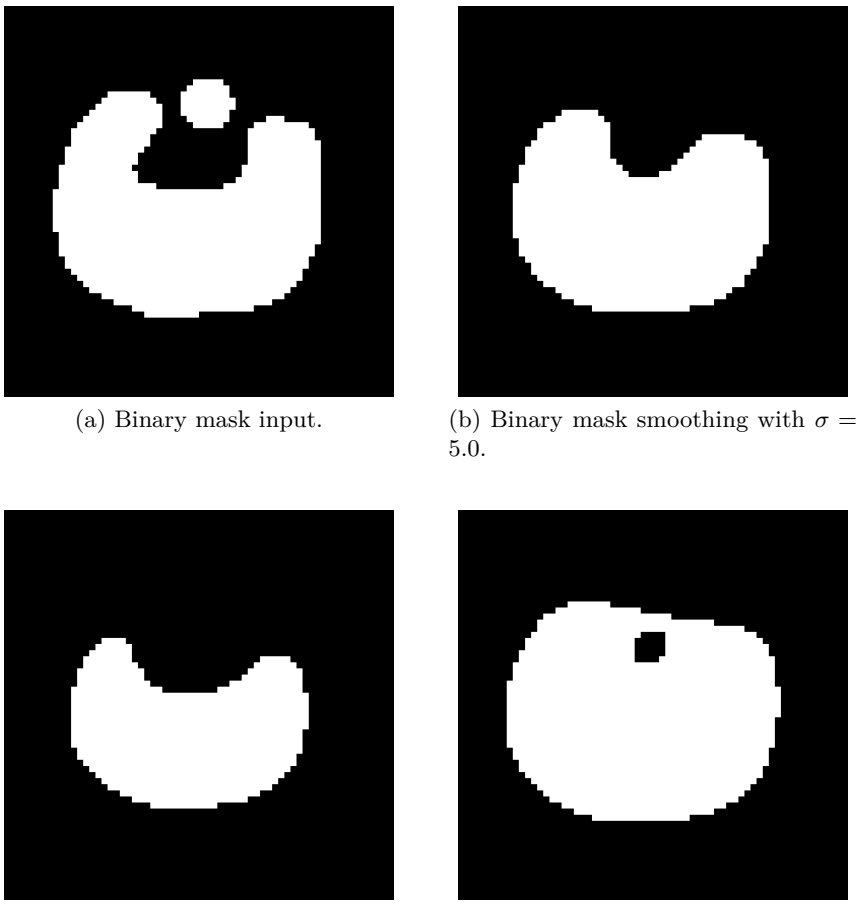
because the value of Gaussian function outside the interval $[-3\sigma, +3\sigma]$ is relatively small.

If we use computed SDF of binary mask as our image, then we get resulting blurred SDF as

$$\mathrm{SDF}_{\mathrm{blurred}}(x,y) = \sum_{u=-R}^{R} \sum_{v=-R}^{R} \mathrm{SDF}(x-u, y-v) G(u,v) \quad , \tag{2.9}$$

where $\mathrm{SDF}(x,y)$ is an element of SDF at x and y coordinate, and $0 \leq x <$ width and $0 \leq y <$ height.

The user is given option to adjust both Gaussian blur parameter σ the threshold value. Increasing the threshold erodes the resulting mask, shown on Fig. 2.3c, whereas decreasing it expands the foreground region, shown on Fig. 2.3d.



(a) Binary mask input.

(b) Binary mask smoothing with $\sigma = 5.0$.

(c) Eroded binary mask with threshold $= 1.5$.

(d) Expanded binary mask with threshold $= -1.0$.

Figure 2.3: Examples of smoothed binary masks. Input binary mask (a) is smoothed to (b) with Gaussian parameter $\sigma = 5.0$. Smoothed binary mask (b) is eroded (c) with threshold $= 1.5$ and is expanded (d) with threshold $= -1.0$.

2.2 Topology Extraction from Binary Masks

2.2.1 Binary Mask Skeletonization

Thinning of a binary mask, referred to as skeletonization, is a process of removing boundary pixels of a foreground region while maintaining their basic structure, general shape and connectivity. For algorithms used to create binary mask skeleton used for this application, we need to ensure satisfaction of certain criteria:

- The resulting line should be 1 pixel wide with no redundant pixels
- Connectivity of individual components must be preserved
- The skeleton should be approximately centered with respect to the local thickness of the component.
- There should not be substantial gap between skeleton and edge of binary mask, if the foreground region is touching the side.

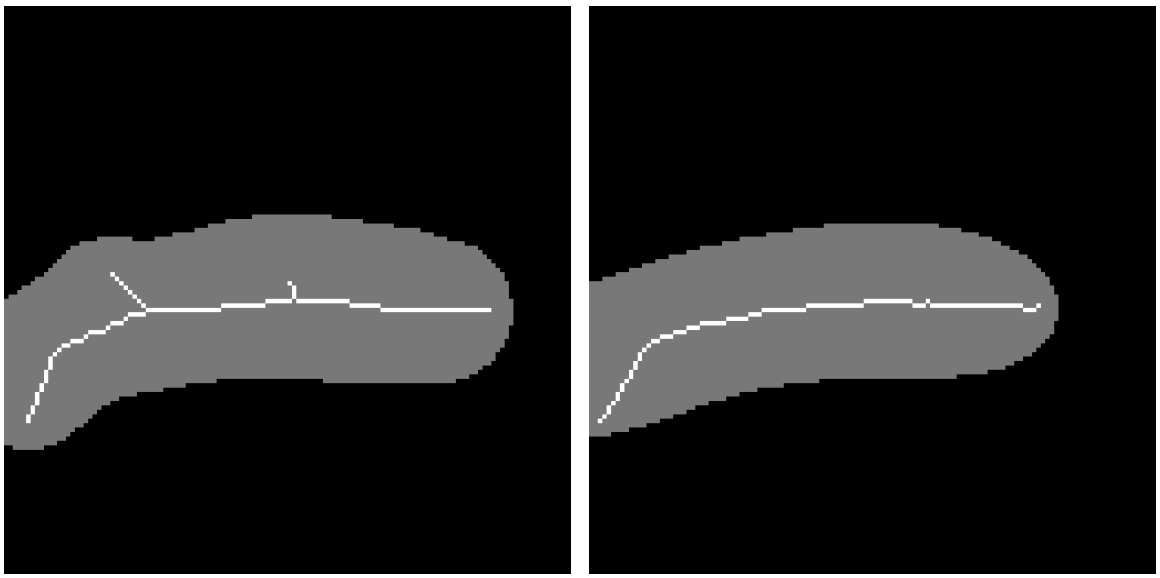
Many thinning algorithms have been proposed for various applications, one of the popular ones are Guo–Hall algorithm [24] and Zhang–Suen algorithm [30]. Their main advantage is their ability to be parallelized, but both of them do not guarantee desired 1-pixel width without some additional post-processing. A different algorithm was selected [18] made by Jain and Kumar, which is sequential and guarantees skeleton with no redundant pixels and 1-pixel width. Difference between each of those algorithms can be seen on Fig. 2.4.

Figure 2.4: Comparison between different skeletonization algorithms. Image was taken from [18]. From top to bottom: (a) Original pattern, (b) Guo-Hall algorithm, (c) Zhang–Suen algorithm, (d) Jain–Kumar algorithm.

This algorithm works by performing passes in all four directions and sequentially removes redundant pixels until only the skeleton remains. This results in the skeleton being offset by 1 pixel from center, but this error is insignificant and does not negatively affect the final result.

Disadvantage of skeletonization algorithms is that they are sensitive to binary mask "corners". As the algorithm creates skeleton to be in the middle, when some section of the foreground region is sharper, it can create branch towards this corner. This can be resolved by smoothing the binary mask before performing skeletonization. Despite the mask slightly changing shape due to the smoothing, the general shape of the skeleton is preserved.

As seen on Fig. 2.5b, even performing skeletonization on smoothed binary mask can lead to undesirable curving on the skeleton. This level of control can not be achieved by simple binary mask smoothing but other more complex algorithms would be needed. However, this results is acceptable for out application of creating splines, as the user has the option to manually adjust them, if the error is too noticeable.



(a) Skeletonization performed on binary mask

(b) Skeletonization performed on smoothed binary mask

Figure 2.5: Example of resulting binary mask skeleton without smoothing (a) and with smoothing (b). The gray area is the binary mask, either initial or smoothed, from which the skeleton was extracted. Smoothing was performed with $\sigma = 15.0$ and threshold -1.2.

2.2.2 Contour Extraction of Area Features

Extracting contours of binary mask foreground region is used for creating features that fill area, such as biomes or lakes.

Used algorithm cycles through every pixel of binary mask. If the pixel is black, that means in background, we skip it. If the pixel is white, in foreground, we count how many 4-neighbors are black and if there are any, we mark this pixel as contour. Pixels that are 4-neighbors are the pixels directly above, below, left or right of the initial pixel.

This algorithm skips foreground pixels along the edges of binary mask, as there are no background pixels. To create edge contour we count pixels that are outside the binary mask bounds as background.

This algorithm, although simple, reliably detects contours. There are a few scenarios where strictly 1-pixel width of the contours is not ensured. These scenarios are hard to resolve by algorithm as their outcome can rapidly change the shape or size of the contours. We decided to leave the resolution of these situations to user by giving them tools for manual edit.

2.3 Polyline Creation from Topological Pixels

A polyline is represented os an ordered list of points $\mathbf{P}_0, \mathbf{P}_1, ..., \mathbf{P}_n$ where each consecutive pair of points $(\mathbf{P}_i, \mathbf{P}_{i+1})$, where $i = 0, 1, ..., n$, defines one line segment. In this way, the polyline forms a connected piecewise curve composed of the segments

$$(\mathbf{P}_0, \mathbf{P}_1), (\mathbf{P}_1, \mathbf{P}_2), ..., (\mathbf{P}_{n-1}, \mathbf{P}_n).$$

Since the points are ordered, the connectivity of the polyline is given implicitly by their order in the list.

Special case of polyline is a closed polyline. A polyline is considered closed if its first and last point are identical,

$$\mathbf{P}_0 = \mathbf{P}_n. \tag{2.10}$$

2.3.1 Pixel Classification

To create polylines we need to classify topological pixels, determining whether a pixel is an endpoint or a junction. We use a simple classification method based on counting the number of foreground pixels in the 8-neighborhood of the examined pixel.

If the number of foreground neighbors is one, the pixel is classified as an endpoint, since it is connected to only one neighboring pixel. If the number is two, the pixel is considered a regular polyline pixel connecting two neighboring pixels.

A junction pixel has three or more foreground neighbors, although in typical cases it has exactly three. If four or more foreground neighbors are present, this may indicate a more complex local structure, such as a plateau or another shape that can complicate polyline extraction.

2.3.2 Polyline Tracing Algorithm

The extracted topology, whether obtained from a binary mask skeleton or from a contour, is still represented as an unordered set of foreground pixels in the image grid. For subsequent processing, such as polyline simplification or spline construction, this discrete representation must be converted into one or more ordered polylines.

The goal of the tracing algorithm is therefore to transform the foreground topology pixels into a set of ordered polylines that collectively cover the entire topology. To achieve this, the algorithm uses the previously detected endpoint and junction pixels together with a visited map. The visited map records whether a foreground pixel has already been assigned to a traced polyline, which prevents repeated processing of the same branch. Junction pixels require special handling, since a single junction may belong to multiple output polylines.

The tracing is divided into three steps. First, all branches starting at endpoints are traced until another endpoint or a junction is reached. Second, the remaining branches connecting pairs of junctions are traced. Finally, any still unvisited foreground pixels are assumed to belong to closed contours and are processed separately. This ordering is important, because endpoint-based branches are the least ambiguous, while closed contours can only be identified reliably after all open branches have already been removed.

When tracing from a current pixel, the algorithm examines its 8-neighborhood and selects the next valid foreground pixel that has not yet been visited. If multiple candidates

exist, endpoint or junction pixels are preferred over regular foreground pixels. Among such candidates, the closest one is selected according to Manhattan distance. This rule improves local continuity of the traced polyline and reduces short redundant detours that may otherwise arise in the discrete pixel representation.

Algorithm 1: Topology tracing into ordered polylines

```
1:  procedure TRACE_TOPOLOGY(foreground_pixels, endpoints, junctions)
2:      visited ← map initialized to false
3:      result ← empty list of polylines
4:
5:      for all endpoint from endpoints do
6:          if not visited[ endpoint ] then
7:              trace a polyline starting at endpoint
8:              stop when another endpoint or a junction is reached
9:              mark regular traced pixels as visited
10:             add the resulting polyline to result
11:         end if
12:     end for
13:
14:     for all junction from junctions do
15:         for all unvisited foreground neighbors of junction do
16:             trace a polyline starting at junction
17:             stop when another junction is reached
18:             mark regular traced pixels as visited
19:             add the resulting polyline to result
20:         end for
21:     end for
22:
23:     for all unvisited foreground pixels p do
24:         trace a closed contour starting at p
25:         continue until no unvisited foreground neighbor exists
26:         append the starting point again to represent closure
27:         add the resulting polyline to result
28:     end for
29:     return result
30: end procedure
```

Tracing from endpoints. In the first stage, each unvisited endpoint is used as a starting point of a new polyline. The tracing then proceeds through unvisited foreground pixels until either another endpoint or a junction is reached. Regular foreground pixels are appended to the polyline and marked as visited. If the traced branch terminates at a junction, the junction is appended to the polyline but is not marked as visited, since it may also serve as a connection point for other branches.

Tracing between junctions. After all endpoint-based branches have been processed, some foreground pixels may still remain unvisited. In a skeleton topology, these pixels are typically expected to form branches connecting one junction to another. Since a single junction may have multiple outgoing branches, each of its unvisited foreground neighbors is considered

separately. For each such neighbor, a new polyline is initialized with the starting junction as its first point and traced until another junction is reached. As in the previous case, only regular foreground pixels are marked as visited during the traversal.

Tracing closed contours. After the previous two stages, any remaining unvisited foreground pixels are assumed to belong to closed contours with no endpoints and no junctions. This case occurs most commonly when tracing contours of foreground regions rather than binary mask skeletons. The tracing starts from any remaining unvisited foreground pixel and continues through unvisited foreground neighbors until the contour has been fully traversed. To represent closure explicitly, the starting point is appended once more at the end of the resulting polyline so that the first and last point are identical.

2.4 Polyline Post-processing

2.4.1 Polyline Stitching Algorithm

Polyline stitching is the process of connecting two or more polylines by merging pairs of endpoints, resulting in one or more longer continuous polylines. A distance threshold can be specified that determines when two endpoints are considered close enough to be merged.

The proposed algorithm follows a greedy strategy. In each iteration, it considers all endpoints of currently active polylines and finds the closest valid pair whose distance does not exceed the specified threshold. The corresponding polylines are then merged into a new polyline, while one of the original polylines is marked as inactive. This process is repeated until no further valid merge can be found.

Algorithm 2: Polyline Stitching algorithm

```
1:  procedure STITCH_POLYLINES(polylines, max_distance)
2:      resulting_polylines ← initialize empty list
3:      used_polylines ← initialize list with false values, length of polylines
4:      polyline_ends ← initialize empty list of points
5:
6:      while true do
7:          polyline_ends.reset()
8:
9:          for polyline from polylines do
10:             if not used_polylines[ polyline ] then
11:                 polyline_ends.add(polyline.first)
12:                 polyline_ends.add(polyline.last)
13:             end if
14:         end for
15:
16:         best_pair ← iniitalize
17:         best_distance = ∞
18:         for first_end from polyline_ends do
19:             for second_end from polyline_ends do
20:                 if if both ends are not from the same polyline then
21:                     distance = ||first_end − second_end||
```

```
22:
23:                     if distance ≤ max_distance and distance < best_distance then
24:                         best_pair = (first_end, second_end)
25:                         best_distance = distance
26:                     end if
27:                 end if
28:             end for
29:         end for
30:         if best_pair not chosen then
31:             break
32:         end if
33:
34:         construct merged_polyline from the best_pair
35:         reverse one polyline if necessary
36:         remove the duplicate connecting endpoint
37:
38:         polylines[ index of best_pair.first ] = merged_polyline
39:         used_polylines[ best_pair.second] = true
40:     end while
41:
42:     for polyline from polylines do
43:         if not used_polylines[ polyline] then
44:             resulting_polylines.add(polyline)
45:         end if
46:     end for
47:     return resulting_polylines
48: end procedure
```

Depending on which endpoints are connected, one of the polylines may need to be reversed before merging in order to preserve a consistent point ordering. When polylines are merged, one of the connecting endpoints must be removed, as it would result in a duplicate point and other algorithms may not work as intended.

In the implementation, an additional iteration limit can be used as a safety measure to prevent unexpected infinite execution in degenerate cases.

2.4.2 Polyline Simplification Using Douglas–Peucker Algorithm

Polyline created from binary mask skeleton or contour can have large amount of redundant points, which could hinder subsequent spline interpolation. To solve this, we employ Douglas–Peucker algorithm [26] for simplifying polylines.

Distance parameter $\epsilon > 0$ is chosen. The algorithm is initially given first and last point of the polyline and both of them are marked to be kept. Line segment is formed between them. It then examines all the remaining points and finds with the maximum distance to this line segment. If this distance is greater than ϵ, the point is marked to be kept and the algorithm is recursively applied to the two sub-polylines defined by the first point, the selected point, and the last point. Example of reduced polyline by Douglas–Peucker algorithm can be seen on Fig. 2.6.

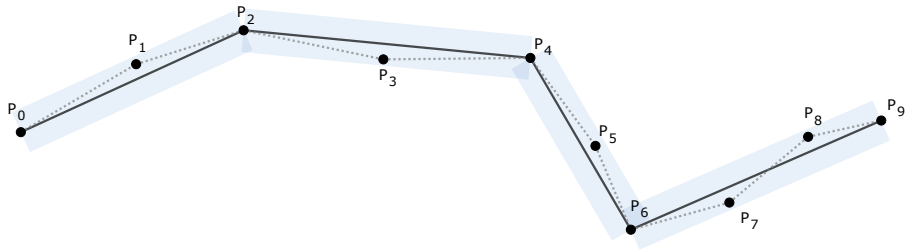


Figure 2.6: Illustration of Douglas–Peucker algorithm. Initial polyline is drawn in gray dotted line, final polyline is drawn in black. Blue rectangles surrounding the polyline is visualization of parameter ϵ.

When the recursion is completed a new polylines is constructed from the points which are marked to be kept. Resulting polyline is of the same orientation and approximates the shape of the initial polyline, depending on the parameter ϵ. This simplification step reduces the number of control points while preserving the overall shape of the extracted topology, which makes the subsequent spline construction more stable and easier to control.

2.5 Spline Construction from Polylines

2.5.1 Hermite and Bézier Curve Representations

In our implementation, we chose to represent splines as a sequence of cubic Bézier curves rather than as Hermite curves, which are used internally by Unreal Engine 5. The main reason for this choice is that Bézier curves are more convenient for the algorithms introduced later in this chapter. At the same time, a cubic Bézier curve can be converted to a Hermite representation in a straightforward manner.

A cubic Bézier curve $\mathbf{B}(t)$, shown on Fig. 2.7, is defined by two end points, $\mathbf{P}_0$ and $\mathbf{P}_3$, and two control points, $\mathbf{P}_1$ and $\mathbf{P}_2$. It is given by

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3 \quad , \tag{2.11}$$

for $t \in [0, 1]$.

A Hermite curve $\mathbf{H}(t)$, shown on Fig. 2.8, is defined by two end points, $\mathbf{H}_0$ and $\mathbf{H}_1$, and two tangent vectors, $\mathbf{M}_0$ and $\mathbf{M}_1$. It is given by

$$\mathbf{H}(t) = (2t^3 - 3t^2 + 1)\mathbf{H}_0 + (t^3 - 2t^2 + t)\mathbf{M}_0 + (-2t^3 + 3t^2)\mathbf{H}_1 + (t^3 - t^2)\mathbf{M}_1 \quad , \tag{2.12}$$

for $t \in [0, 1]$.

A Hermite curve defined by end points $\mathbf{H}_0$, $\mathbf{H}_1$ and tangent vectors $\mathbf{M}_0$, $\mathbf{M}_1$ can be constructed from a cubic Bézier curve defined by control points $\mathbf{P}_0$, $\mathbf{P}_1$, $\mathbf{P}_2$, and $\mathbf{P}_3$ using the

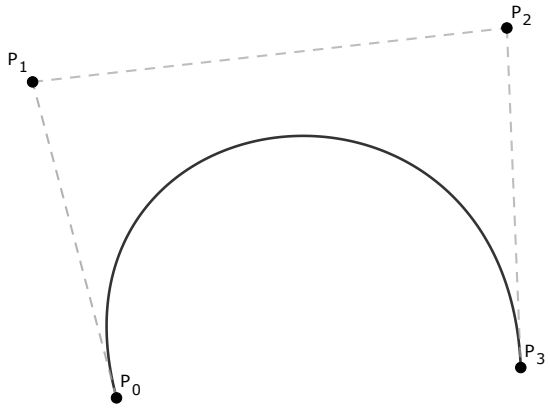


Figure 2.7: Example of cubic Bézier curve constructed from four control points $\mathbf{P_0}$ to $\mathbf{P_3}$.

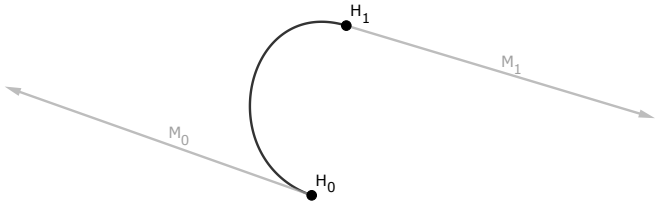


Figure 2.8: Illustration of Hermite spline constructed from two points $\mathbf{H_0}$,$\mathbf{H_1}$ and two tangent vectors $\mathbf{M_0}$,$\mathbf{M_1}$.

following relationships:

$$\begin{aligned} \mathbf{H}_0 &= \mathbf{P}_0 \quad , \\ \mathbf{M}_0 &= 3\left(\mathbf{P}_1 - \mathbf{P}_0\right) \quad , \\ \mathbf{M}_1 &= 3\left(\mathbf{P}_3 - \mathbf{P}_2\right) \quad , \\ \mathbf{H}_1 &= \mathbf{P}_3. \end{aligned} \tag{2.13}$$

2.5.2 Catmull–Rom Spline Construction

To interpolate spline through points of a polyline, we employed a Catmull–Rom spline. Catmull–Rom splines are a family of cubic interpolating splines constructed such that a tangent at point $\mathbf{P}_i$ is computed using the previous point $\mathbf{P}_{i-1}$ and the next point $\mathbf{P}_{i+1}$.

We used a class of parameterization described in [25] of Catmull–Rom spline. Let the knot sequence satisfy

$$t_{k+1} = t_k + \|\mathbf{P}_{k+1} - \mathbf{P}_k\|^{\alpha} \quad , \tag{2.14}$$

where $\alpha \in [0, 1]$ and $t_0 = 0$. If $\alpha = 0$ then the parameterization is uniform, for $\alpha = 1$ the parameterization becomes chordal. Example of spline interpolation through polyline defined by points $\mathbf{P_0}$ to $\mathbf{P_5}$ can be seen on Fig. 2.9. On this figure, it can be observed that the spline interpolation does not connect first and last points. This property can also be observed in the definition 2.14.

In our implementation, we use $\alpha = 0.5$, which corresponds to the centripetal parameterization of Catmull–Rom spline. This variant was selected because it is the only parameterization in this family that guarantees no cusps or self-intersections within a single curve segment [29].

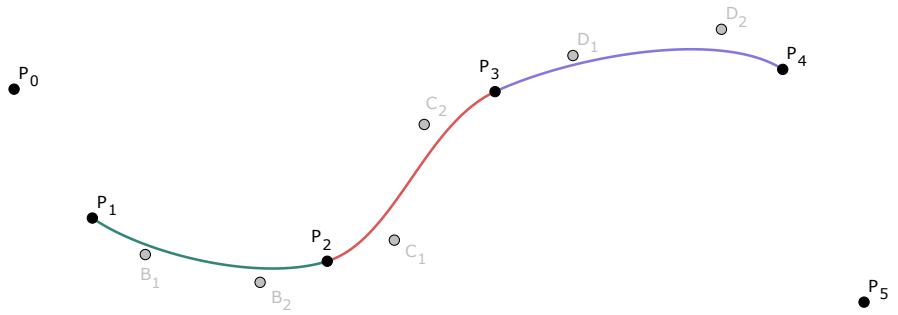


Figure 2.9: Example of spline interpolation using Catmull–Rom spline through polyline of six points, $\mathbf{P_0}$ to $\mathbf{P_5}$ are corresponding with corresponding control points for Bézier curve colored in gray.

As follows from the definition, a Catmull–Rom spline segment is constructed from four points, $\mathbf{P}_0$ to $\mathbf{P}_3$, and produces a curve segment between $\mathbf{P}_1$ and $\mathbf{P}_2$. For an open polyline, this means that the spline would not naturally interpolate the first and the last point of the

polyline. To resolve this issue, we introduce auxiliary end points defined as

$$\begin{aligned} \mathbf{P}_{-1} &= 2\mathbf{P}_0 - \mathbf{P}_1 \quad , \\ \mathbf{P}_{n+1} &= 2\mathbf{P}_n - \mathbf{P}_{n-1} \quad , \end{aligned} \tag{2.15}$$

where the polyline consists of points $\mathbf{P}_0, \ldots, \mathbf{P}_n$. Example of such extension can be seen on Fig. 2.10.

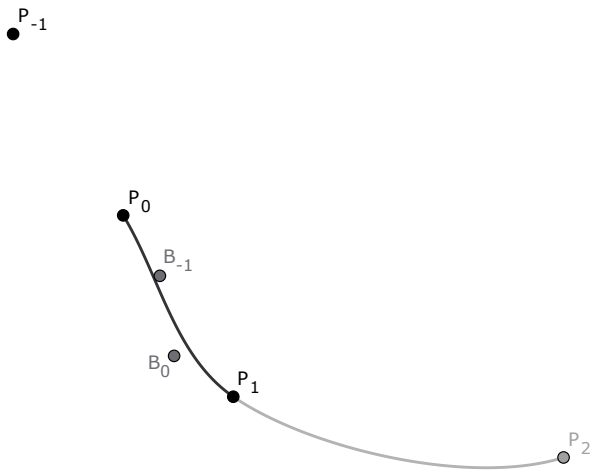


Figure 2.10: Illustration of Catmull–Rom spline endpoint extension using auxiliary point $\mathbf{P_{-1}}$.

A drawback of this endpoint extension is that it may produce undesirable shapes when the polyline forms a sharp turn near the boundary. As shown in Fig. 2.11, the first spline segment is constructed from the points $\mathbf{P}_{-1}, \mathbf{P}_0, \mathbf{P}_1$, and $\mathbf{P}_2$. Since $\mathbf{P}_{-1}$ is defined to be collinear with $\mathbf{P}_0$ and $\mathbf{P}_1$, the tangent of the segment between $\mathbf{P}_0$ and $\mathbf{P}_1$ at $\mathbf{P}_0$ is aligned with the line passing through these points. As a result, the curve segment is pulled in the tangent direction, which may lead to an unnatural shape near the endpoint. However, since the base polylines are constructed from binary mask skeletons or contours, this situation occurs only rarely in practice.

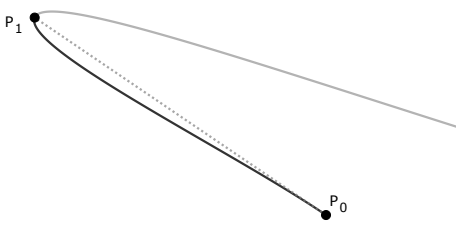


Figure 2.11: Illustration of Catmull–Rom endpoint extension sharp curve resulting in unnatural curve near the endpoint $\mathbf{P_0}$.

The knot sequence in Equation (2.14) determines the parameter spacing between interpolation points. Unlike the Hermite or Bézier formulations, it does not directly provide a

polynomial expression of the curve segment. However, this parameterization can be used to construct equivalent Hermite or Bézier representations of the same curve segment.

The Bézier curve representation of a Catmull–Rom spline segment can be computed directly from the four defining points, $\mathbf{P}_0$ to $\mathbf{P}_3$. If the Bézier curve is defined by control points $\mathbf{B}_0$ to $\mathbf{B}_3$, the relationship is given by [29]

$$\begin{aligned}
\mathbf{B}_0 &= \mathbf{P}_1 \quad , \\
\mathbf{B}_1 &= \frac{d_1^{2\alpha}\mathbf{P}_2 - d_2^{2\alpha}\mathbf{P}_0 + \left(2d_1^{2\alpha} + 3d_1^{\alpha}d_2^{\alpha} + d_2^{2\alpha}\right)\mathbf{P}_1}{3d_1^{\alpha}\left(d_1^{\alpha} + d_2^{\alpha}\right)} \quad , \\
\mathbf{B}_2 &= \frac{d_3^{2\alpha}\mathbf{P}_1 - d_2^{2\alpha}\mathbf{P}_3 + \left(2d_3^{2\alpha} + 3d_3^{\alpha}d_2^{\alpha} + d_2^{2\alpha}\right)\mathbf{P}_2}{3d_3^{\alpha}\left(d_3^{\alpha} + d_2^{\alpha}\right)} \quad , \\
\mathbf{B}_3 &= \mathbf{P}_2 \quad ,
\end{aligned} \tag{2.16}$$

where d_1, d_2, and d_3 are defined as

$$\begin{aligned}
d_1 &= \|\mathbf{P}_1 - \mathbf{P}_0\| \quad , \\
d_2 &= \|\mathbf{P}_2 - \mathbf{P}_1\| \quad , \\
d_3 &= \|\mathbf{P}_3 - \mathbf{P}_2\| \quad ,
\end{aligned} \tag{2.17}$$

and $\|\cdot\|$ denotes the Euclidean norm. The knot intervals corresponding to the distances d_1, d_2, and d_3 satisfy

$$\begin{aligned}
t_1 - t_0 &= d_1^{\alpha} \quad , \\
t_2 - t_1 &= d_2^{\alpha} \quad , \\
t_3 - t_2 &= d_3^{\alpha}.
\end{aligned} \tag{2.18}$$

2.5.3 Spline Subdivision Using de Casteljau's Algorithm

Bézier curve can be constructed recursively by using de Casteljau's algorithm. This method is numerically more stable, than using polynomial definition (2.11). Another important use case of this algorithm is subdividing Bézier curve into two segments and subsequently getting correct their control points.

Let cubic Bézier curve $\mathbf{B}(t)$ be defined by four control points $\mathbf{P}_0$ to $\mathbf{P}_3$ and $t \in [0, 1]$. De Casteljau's algorithm for cubic Bézier curve $\mathbf{B}$ works by performing three steps. For parameter $u \in [0, 1]$ it creates points

$$\mathbf{Q}_i = (\mathbf{P}_{i+1} - \mathbf{P}_i)u + \mathbf{P}_i \quad , \tag{2.19}$$

for $i = 0, 1, 2$. Point $\mathbf{Q}_i$ is a point on a line connecting control points $\mathbf{P}_i$ an $\mathbf{P}_{i+1}$. Then it creates another points

$$\mathbf{R}_j = (\mathbf{Q}_{j+1} - \mathbf{Q}_j)u + \mathbf{Q}_j \quad , \tag{2.20}$$

for $j = 0, 1$. Similarly as with $\mathbf{Q}_i$, point $\mathbf{R}_j$ is a point on a line connecting points $\mathbf{Q}_j$ and $\mathbf{Q}_{j+1}$. Final point is constructed on a line between $\mathbf{R}_0$ and $\mathbf{R}_1$ as

$$\mathbf{S} = (\mathbf{R}_1 - \mathbf{R}_0)u + \mathbf{R}_0. \tag{2.21}$$

Final point $\mathbf{S}$ lies on a cubic Bézier $\mathbf{B}(t)$ and is equal to a point $\mathbf{B}(u)$. Algorithm is visualized on Fig. 2.12

We can then create two cubic Bézier curves $\mathbf{B}_A(t_A)$ and $\mathbf{B}_B(t_B)$, and $t_A, t_B \in [0, 1]$. Bézier curve $\mathbf{B}_A$, visualized in Fig. 2.13a, is defined by control points $\mathbf{P}_0$, $\mathbf{Q}_0$, $\mathbf{R}_0$ and $\mathbf{S}$. Bézier curve $\mathbf{B}_B$, visualized in Fig. 2.13b, is defined by control points $\mathbf{S}$, $\mathbf{R}_1$, $\mathbf{Q}_2$ and $\mathbf{P}_3$. These segments when combined results in the initial Bézier curve $\mathbf{B}(t)$.

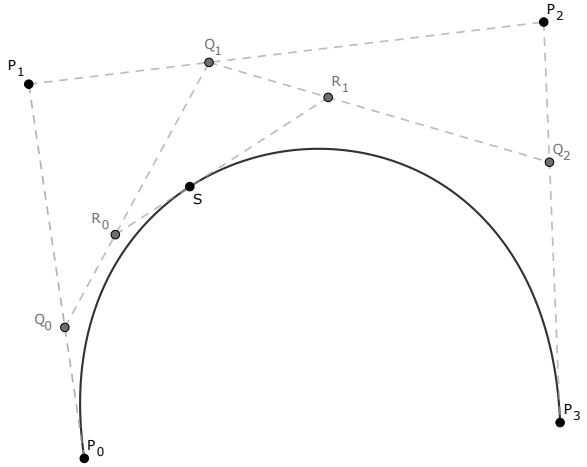
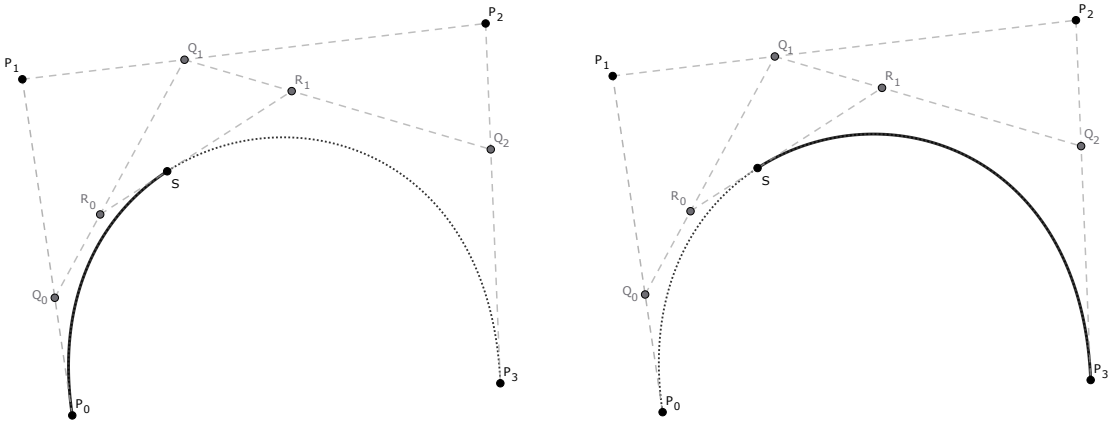


Figure 2.12: Visual illustration of de Casteljau algorithm for $u = 0.35$.



(a) Left spline subdivision.

(b) Right spline subdivision.

Figure 2.13: Example of subdividing Bézier curve using de Casteljau algorithm for $u = 0.35$, the left subdivision resulting in one Bézier curve (a), and the right subdivision resulting in second Bézier curve (b).

Chapter 3

Method Design and Implementation

In the previous chapter, the algorithms and methodology used in the proposed method were described. This method was implemented as Unreal Engine 5 plugin. The plugin depends on several official Unreal Engine plugins, namely Water, PCG Framework [28], and PCG Biome Core [27]. The latter two provide built-in support for procedural generation, while the Water plugin enables correct landscape deformation for rivers and lakes. Plugin was build in Unreal Engine 5 version 5.4.4.

The plugin was designed as an editor plugin, which means that it relies on tools and functionality that are not available at Unreal Engine runtime. As a result, the generated environments must either be compiled into the simulator or loaded from external data prepared in advance.

3.1 Processing Pipeline

The intention of the processing pipeline was its simplicity. User can easily create landscape splines, rivers, lakes and biomes, without deeper knowledge of the Unreal Engine with relative ease. Many of the provided functions are based on existing Unreal Engine tools, but are simplified or wrapped to make them easier to use.

Plugin expects three input files:

- Height map - a square 16-bit gray scale image, where the darkest pixel corresponds to the lowest elevation and the brightest pixel corresponds the highest elevation.
- Segmentation - a square RGB image in which each color represents different biome or region.
- Segmentation legend - a JSON file in which each entry contains three parameters: region name, region color, and region type. The currently implemented region types are BIOME, RIVER, LAKE, and ROAD.

The distinction between these region types is necessary because each type corresponds to a different generation workflow and a different type of Unreal Engine object.

The user can then create the landscape, landscape splines representing roads, river and lake splines, as well as biome splines with basic control over PCG Biome Core. The plugin provides a fast and accessible workflow for environment creation, resulting in a substantial reduction in the time required to prepare a level. After generation, the user can further refine the result using built-in Unreal Engine tools.

The plugin is organized into several pages, each corresponding to a specific workflow within Unreal Engine. These pages are:

- **Startup page** - loading of all input files.
- **Landscape page** - creation of a landscape from a height map.

- **Road spline page** - creation of landscape splines for roads.
- **Water body page** - creation of river and lake splines.
- **Biome page** - creation of PCG Biome Core objects as well as biome splines.

The Road Spline, Water Body, and Biome pages share a common part of the processing pipeline, namely segmentation processing and spline calculation. Additionally, possible introduction of other Unreal Engine 5 objects in form of splines is made more accessible thanks to the common part and underlying spline representation.

3.2 Landscape Creation

This plugin simplifies the process of landscape creation. Its functionality is similar to the built-in Unreal Engine landscape creation workflow. It also includes height map upscaling, but it requires a square-shaped height map. During upscaling, the user can choose the target resolution from a predefined set of valid values. The resolutions used for landscape generation in Unreal Engine 5 must satisfy the following requirements:

$$\begin{aligned} W - 1 &\equiv 0 \pmod{S_Q} \quad , \\ H - 1 &\equiv 0 \pmod{S_Q} \quad , \end{aligned} \tag{3.1}$$

where quad size of component $S_Q = N_S \times S_S$, S_S is quad size of one subsection, and N_S is number of subsections. Width and height of the height map are represented as W and H.

Upscaling of the landscape height map is performed using bilinear interpolation. For each pixel of the destination height map, the corresponding continuous position in the source height map is computed. Since this position usually lies between four neighboring source pixels, the resulting height value is obtained by interpolating between these four samples according to their relative distances.

The scaling factors are chosen so that the boundary pixels of the source and destination heightmaps remain aligned. After interpolation, the computed value is rounded, clamped to the valid 16-bit range, and stored in the destination heightmap. This produces a smoother result than nearest-neighbor upscaling, although it does not introduce any new terrain detail.

A landscape generated directly from a height map is limited by the minimum and maximum values present in that height map. Consequently, the terrain cannot be deformed above or below this range without additional preprocessing. This becomes problematic when generating rivers and lakes, since these features deform the landscape surface.

This is achieved by rescaling all height values so that the terrain includes a user-defined extension above the highest point and below the lowest point. After this adjustment, the scale in the Z axis is recalculated so that the resulting landscape retains the correct real-world size

The remaining settings are equivalent to those of the built-in landscape creation tool, including position, rotation, scale, and the material assigned to the generated landscape. Landscape page of the developed plugin is shown on Fig. 3.1.

3.3 Segmentation Processing

The initial step in the creation of all spline-based Unreal Engine objects is segmentation processing. In this step, the plugin applies the algorithms introduced in Chapter 2 to process

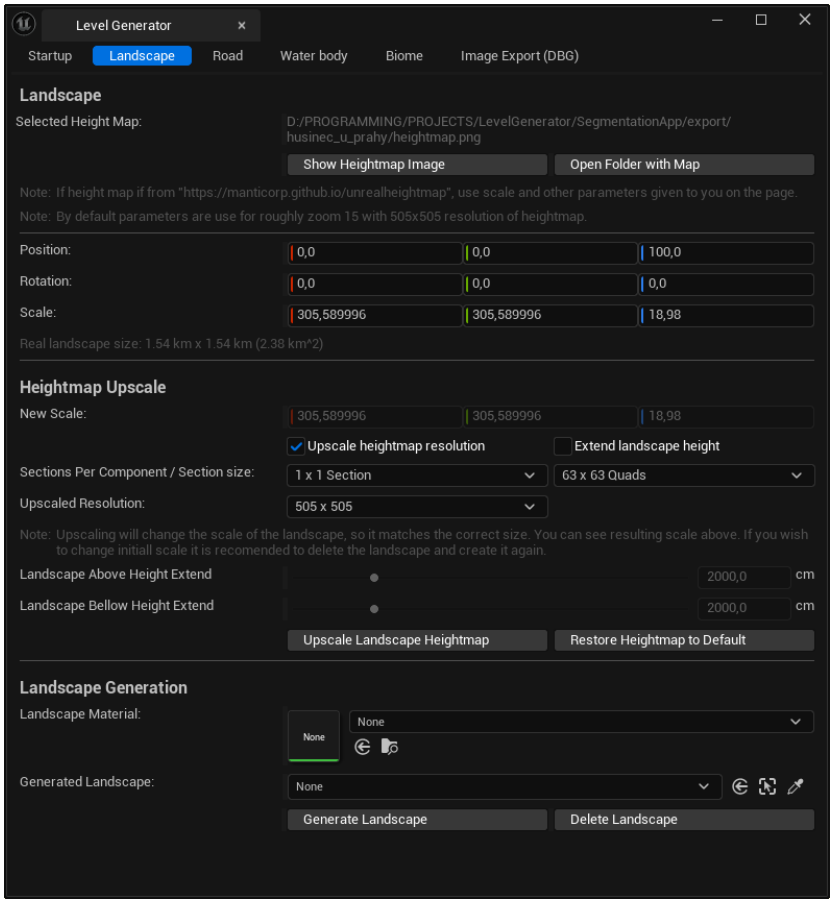
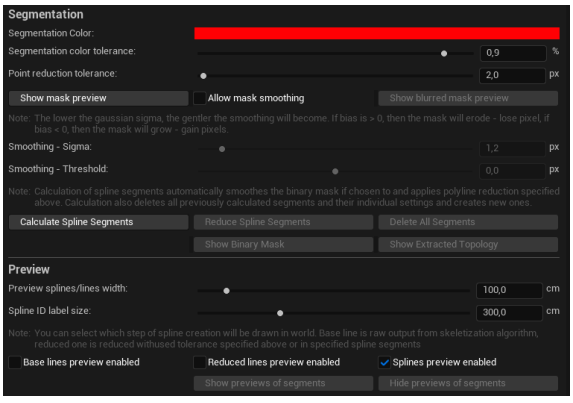
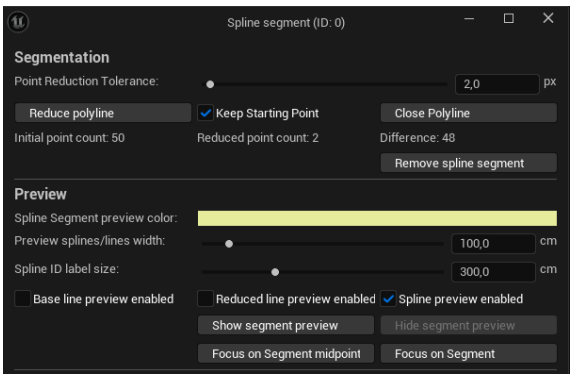


Figure 3.1: Landscape page of the plugin user interface. The page allows user to upscale landscape height map, set landscape properties and material, and landscape creation.



(a) Segmentation page part of all splines.



(b) Segmentation page part of individual spline segment.

Figure 3.2: Figure (a) shows plugin user interface page part of spline creation from segmentation image. Figure (b) shows plugin user interface page part of individual spline segment adjustment based on already generated splines in (a).

the segmentation image. This includes segmentation-based binary mask extraction, optional binary mask smoothing, polyline simplification using the Douglas–Peucker algorithm, and subsequent spline interpolation. The plugin also provides spline preview functionality, allowing the user to inspect the generated splines and adjust the parameters accordingly.

The 2D spline, calculated either from a binary mask skeleton or from a contour, is then transformed from binary mask space to landscape space. This process is performed using the Hermite spline representation rather than the Bézier representation, since Hermite splines are more convenient for this type of transformation, as seen on Fig. 3.3.

To transform a point from image space to landscape space, the x and y coordinates are first mapped according to Algorithm 3.

Algorithm 3: Point transformation from image space to landscape space

procedure TRANSFORM_POINT(*image_point*, *image*, *landscape*)
  min_x, min_y, max_x, max_y = *landscape*.get_extends()

  u = *image_point*.x/(*image*.width − 1)
  v = *image_point*.y/(*image*.height − 1)

  $q_x = min_x + (max_x - min_x) \cdot u$
  $q_y = min_y + (max_y - min_y) \cdot v$
  return $(q_x, q_y, 0)$
end procedure

After this planar transformation, a ray parallel to the z-axis is cast from the transformed point. The intersection of this ray with the landscape determines the final spline point in landscape space. In Unreal Engine 5, this operation also provides the landscape normal vector at the point of intersection.

The landscape normal vector $\mathbf{n}_i$ is then used to project the Hermite tangent vector $\mathbf{M}_i$ onto the tangent plane of the landscape surface. For each spline point, the corrected tangent is computed as

$$\mathbf{M}'_i = \mathbf{M}_i - (\mathbf{M}_i \cdot \mathbf{n}_i)\mathbf{n}_i, \tag{3.2}$$

where $i \in \{0, 1\}$ for a single Hermite curve segment.

Fig. 3.2a shows the segmentation-processing part shared by all plugin pages that generate splines. In addition to global processing parameters, the plugin also allows individual adjustment of already generated spline segments, as shown in Fig. 3.2b.

3.4 Unreal Engine Spline Creation Pages

3.4.1 Landscape Spline Creation

The purpose of this page is to create landscape splines on a previously generated landscape. All segmentation regions of type ROAD are displayed in the user interface as a list of collapsible sections, as shown in Fig. 3.4a. Each section contains a segmentation processing part and a landscape spline creation part. User can adjust individual generated splines as shown in Fig. 3.4b.

The segmentation processing part was described in detail in Sec. 3.3. In addition, the page contains a landscape spline configuration part. In this section, the user can specify the

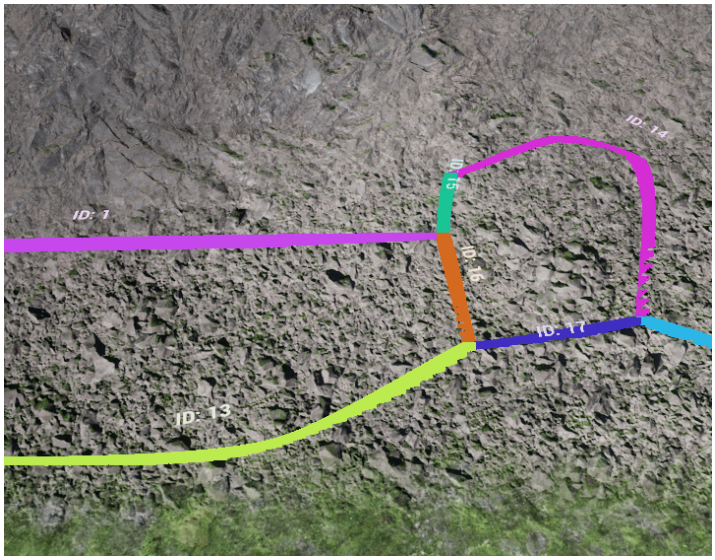


Figure 3.3: Example of spline preview functionality. Identification numbers of each spline can be seen above them. The width of spline preview and size of the ID text can be adjusted in the plugin user interface.

width of the landscape spline and its falloff distance. The width can be determined automatically from the EDT of the binary mask skeleton or specified manually by the user. The falloff distance defines the distance from the spline at which landscape deformation begins.

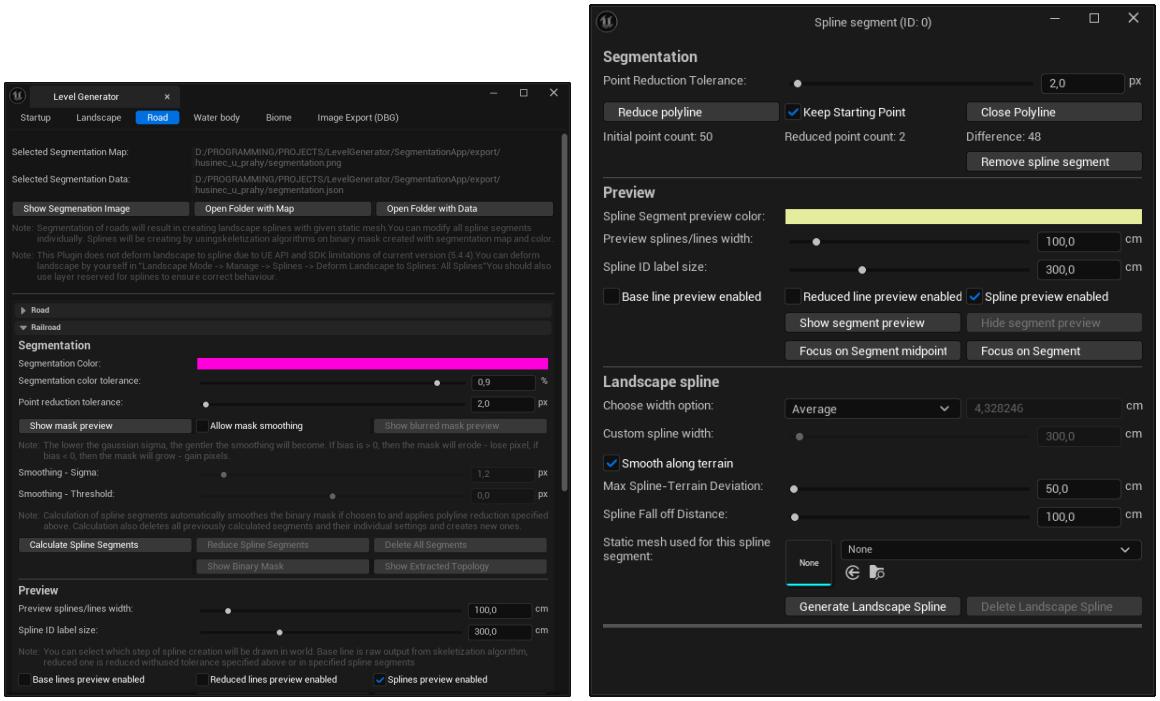
The user can enable an option to smooth the landscape spline with respect to the terrain surface. This operation uses de Casteljau's algorithm, described in Sec. 2.5.3. The smoothing algorithm evaluates the midpoint of a spline segment and casts a ray parallel to the z-axis in order to find its intersection with the landscape. If the distance between the original midpoint and the intersection point exceeds a user-defined threshold, the spline segment is split into two segments and the midpoint is projected onto the landscape as described in Sec. 3.3. This process is repeated until the distance for every segment is below the specified threshold. Result of such smoothing can be seen on Fig. 3.5

In Unreal Engine 5, version 5.5 and later, automatic deformation of the landscape according to the landscape spline is supported through the C++ Application Programming Interface (API). However, this project was developed in Unreal Engine 5 version 5.4.4, where this functionality was not available. For this reason, the plugin provides the user with detailed instructions for performing this step manually.

Example of landscape spline generation with landscape deformation is illustrated on Fig. 3.6. In this example it can be observed that small manual adjustments are needed to achieve more realistic junctions. Landscape deformation is dependent on landscape resolution.

3.4.2 River and Lake Spline Creation

The purpose of this page is to create river or lake splines on a previously generated landscape. Landscape height extension is recommended, as the depth of rivers and lakes can extend bellow the lowest point of the landscape. Similarly to Sec. 3.4.1, all segmentation regions of types RIVER or LAKE are displayed in the user interface as a list of collapsible



(a) Landscape spline page with two segmentation regions of type ROAD: Road and Railroad.

(b) Landscape spline page for individual spline

Figure 3.4: Figure (a) shows plugin user interface page part of a landscape spline creation from segmentation image. Figure (b) shows plugin user interface page part of individual spline segment adjustment based on already generated splines in (a).

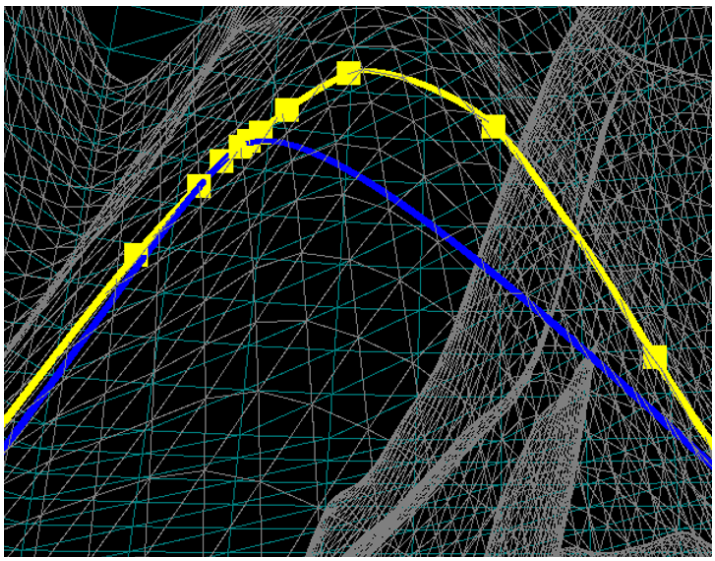


Figure 3.5: Example of spline smoothing with respect to the terrain functionality. Blue spline is the initial, not smoothed out, spline. Yellow spline is smoothed with respect to the terrain. Yellow rectangles are the added spline points needed to achieve this result.

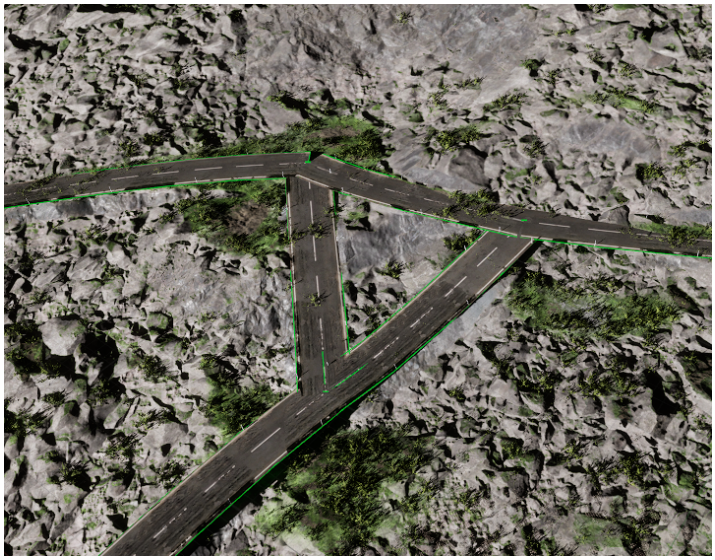


Figure 3.6: Example of generated landscape spline from binary mask skeleton using Catmull–Rom splines with landscape deformation.

sections, shown in Fig. 3.7a. Each section contains a segmentation processing part and a water body part. User can adjust individual generated splines as shown in Fig. 3.7b.

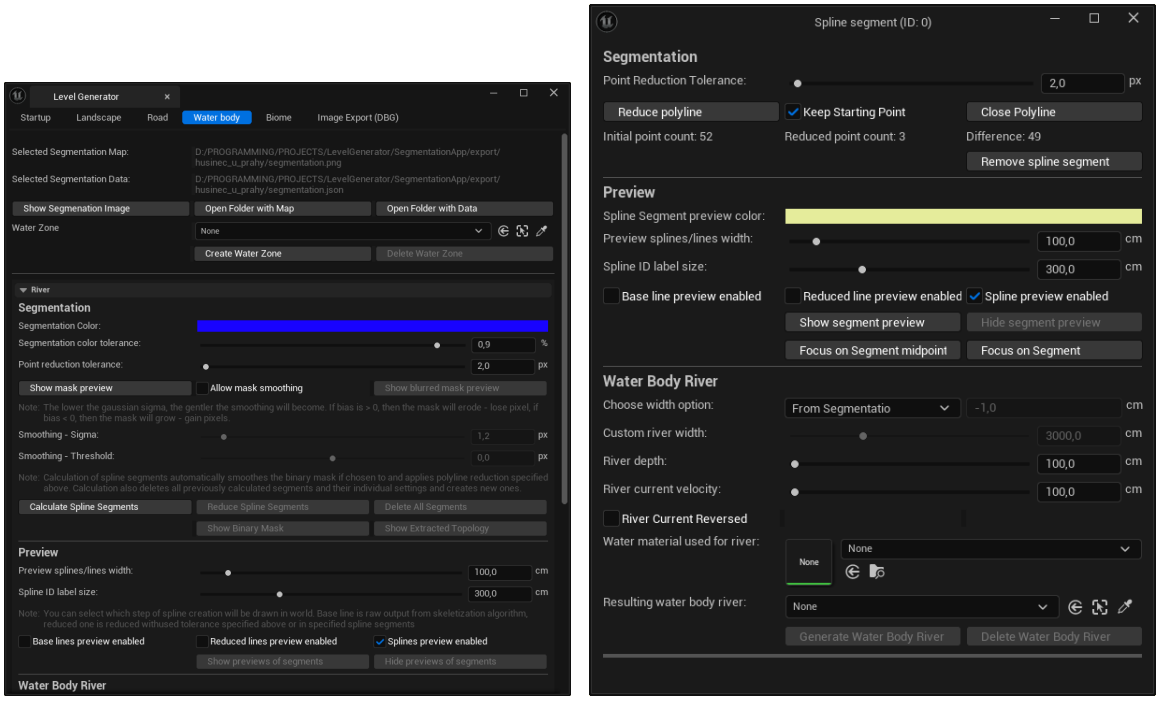
Water zone must be created before user can create water body splines. Water zone encapsulates the entire landscape and its vertical extends. This object is needed for correct creation of water body splines and its usage. Before calculating splines it is recommended to perform binary mask smoothing, otherwise many spline branches can be generated, as seen in Fig. 2.5a. If the binary mask skeleton or outline is generated fragmented even after the binary mask is smoothed out, user can choose to stitch many spline segments together to form one continuous spline or to close opened splines.

When creating river splines, user has the option to adjust depth of the river and the speed and direction of water flow. There is also a option to select river width at each point of the spline accordingly to calculated EDT of the river binary mask region. When creating lake splines, user has the option to only adjust the depth of the lake. Example of river spline is shown on Fig. 3.8.

3.4.3 PCG Biome Core Creation

The Biome page follows the workflow of the PCG Biome Core system [27]. The user first needs to create a PCG Biome Core object. The plugin user interface exposes only the most relevant settings, such as the PCG Graph, which controls how the procedural generation is performed, the biome blending range, and the cache cell size. Increasing the cache cell size may help reduce the performance overhead of the PCG Biome Core system.

The plugin user interface also provides controls for managing the PCG Biome Core object. The user can manually generate, update, or refresh biomes, as well as clean up previously generated biome instances. When working with large and complex biomes, it is recommended to disable automatic refresh of the PCG Biome Core object, since frequent updates may overload the Unreal Engine editor and lead to instability. The available settings are shown in Fig. 3.9a.

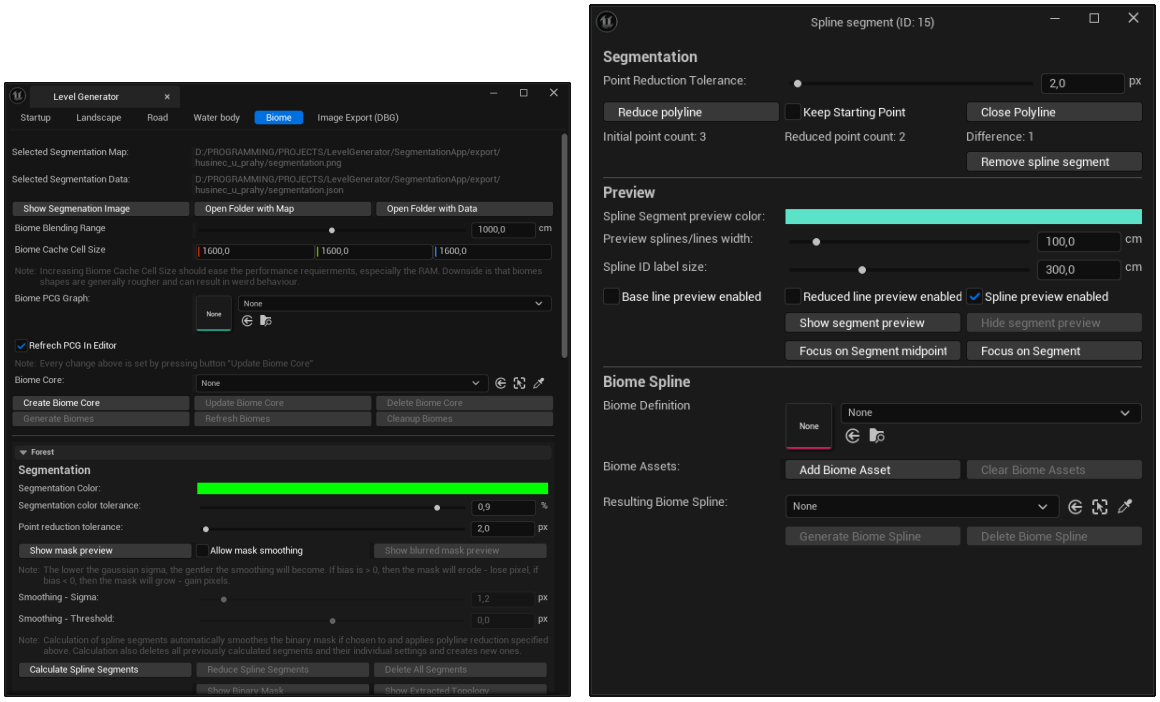


(a) Water body page with region of type RIVER: River. (b) Water body page for individual spline

Figure 3.7: Figure (a) shows plugin user interface page part of a water body spline creation from segmentation image. Figure (b) shows plugin user interface page part of individual spline segment adjustment based on already generated splines in (a).

Figure 3.8: Example of generated river spline from binary mask skeleton using Catmull–Rom splines. Width at each point of the spline was calculated using EDT to achieve more realistic river shape.

The PCG Biome Core system is computationally demanding, especially in terms of memory usage. During testing, large biome setups occasionally led to instability of the Unreal Engine 5 editor. Several mitigation strategies were tested, but none of them significantly improved the observed behavior.



(a) Biome page with two segmentation region of type BIOME: Forest.

(b) Biome page for individual spline

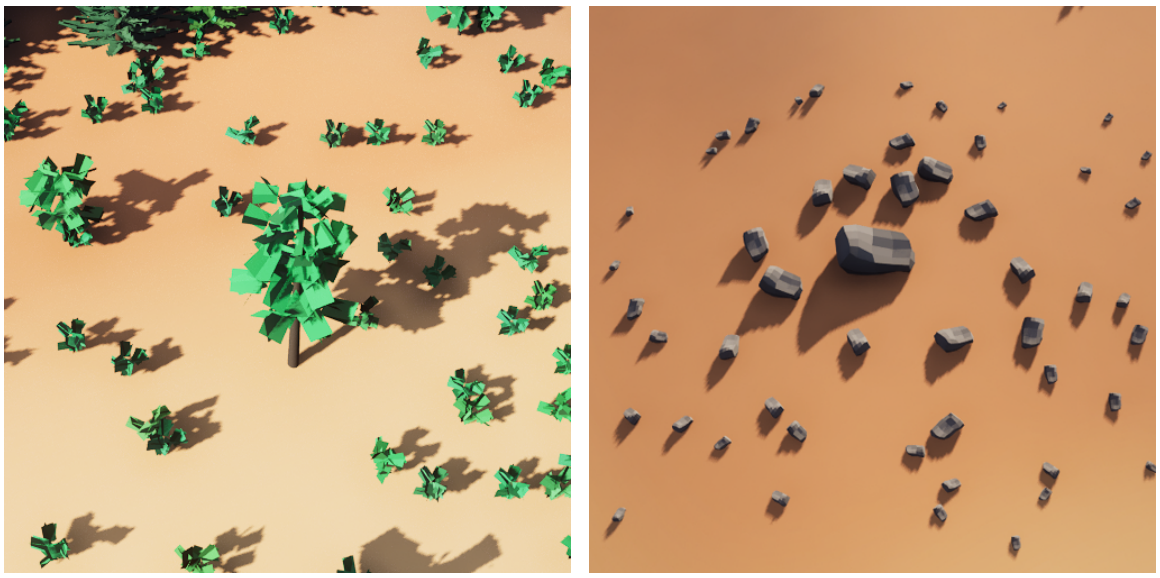
Figure 3.9: Figure (a) shows plugin user interface page part of a biome spline creation from segmentation image. Figure (b) shows plugin user interface page part of individual spline segment adjustment based on already generated splines in (a).

3.4.4 Biome Spline Creation

Biome spline generation requires a Biome Definition and one or more Biome Assets, as shown in Fig. 3.9b. Both are based on the Data Asset type in Unreal Engine 5 and are provided by the PCG Biome Core plugin. The Biome Definition specifies the name and color of the biome. This step is required by all biome construction methods available in the PCG Biome Core plugin, and it is especially important when biomes are generated from a biome mask. For biome spline generation alone, however, the biome name and color are not used directly. Generated biome using biome spline can be seen on Fig. 3.11.

A Biome Asset defines how the biome should be generated. This Data Asset contains settings for the static meshes used in the biome, such as trees and rocks, together with their configuration, probability weights, collision settings, the generator represented by a PCG Graph, optional child biome assets, and other parameters. Child Biome Assets work similarly to their parent assets, but only within a limited distance from the parent asset instances. This makes it possible to create more realistic groupings of generated objects, such as a tree surrounded by saplings, as on Fig. 3.10a or a large rock surrounded by smaller stones, as on

Fig. 3.10b.



(a) Tree surrounded by saplings generated through PCG and Biome Asset.

(b) Rock surrounded by stones generated through PCG and Biome Asset

Figure 3.10: Example of PCG generation through Biome Asset of tree surrounded by saplings (a) and rock surrounded by stones (b).

The generating PCG Graph directly affects how biome objects are distributed. This functionality is provided by the PCG Framework plugin [28]. A typical PCG Graph first generates a large number of random points inside an enclosed area, which in this case is defined by the biome spline. These points are assigned random rotations and scales. The graph then processes the points through a sequence of filters defined either in the PCG Graph itself or in the Biome Asset. These filters modify the number of points or their properties. In the final step, static meshes are spawned at the resulting point locations. This allows the user to prepare different mesh placement procedures depending on the intended biome appearance.

3.5 Generated Worlds

As a showcase of the proposed method, four unique worlds were generated. The environments were chosen to be visually and structurally distinct in order to demonstrate the range of scenarios that can be created using the proposed method. The height maps for all four worlds were generated using an online height map generator for Unreal Engine 5[1], while the segmentation images were created manually.

Following four worlds were generated:

- Desert Oasis - shown on Fig. 3.12.
- Étretat Cliffs - cliffs in northern France shown on Fig. 3.13.
- Grand Canyon - canyon in America shown on Fig. 3.14.
- Stuðlagil Canyon - canyon in Iceland shown on Fig. 3.15.

[1]Height maps were generated using: https://manticorp.github.io/unrealheightmap/.

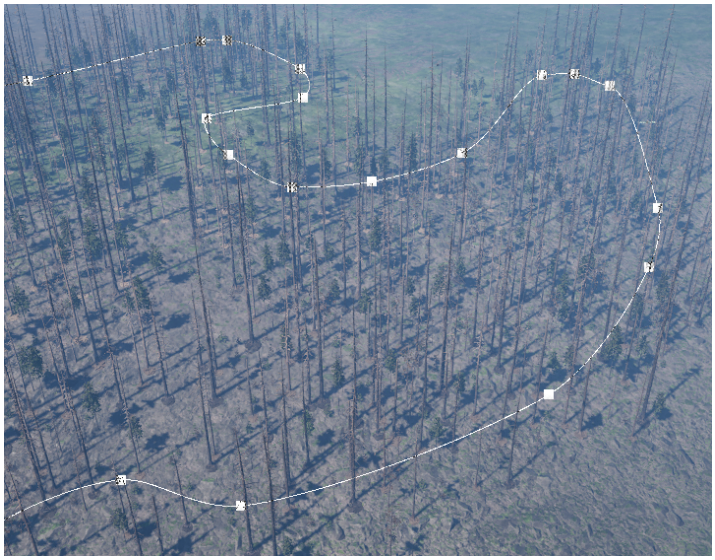


Figure 3.11: Example of generated forest using biome spline in Unreal Engine 5 plugin PCG Biome Core.

Slight manual adjustments were made to correct issues caused by inaccurate height maps. This problem was most noticeable in the Étretat Cliffs world, where the height map had approximately half the resolution of the other height maps. Additionally, river splines were adjusted manually, since the hand-drawn segmentation images did not always precisely follow the intended river paths.

Even though small manual adjustments were required, the time needed to generate each environment was approximately half an hour. This is significantly less than the time that would be required to create similar worlds entirely manually. Most of the time was spent drawing the segmentation images and preparing the PCG Biome Core Data Assets.

(a) (b)

Figure 3.12: World named Desert Oasis generated using proposed method implemented as Unreal Engine 5 plugin.

(a) (b)

Figure 3.13: World named Étretat Cliffs generated using proposed method implemented as Unreal Engine 5 plugin.

(a) (b)

Figure 3.14: World named Grand Canyon generated using proposed method implemented as Unreal Engine 5 plugin.

(a) (b)

Figure 3.15: World named Stuðlagil Canyon generated using proposed method implemented as Unreal Engine 5 plugin.

Chapter 4

Dataset Generation and Verification

Full autonomy of UAVs remains an unsolved problem, and opportunities to evaluate autonomous systems in challenging real-world environments are limited. One possible approach is to use procedurally generated environments to test autonomous methods under controlled but challenging conditions before deploying them in the real world.

To demonstrate the proposed method on generating large-scale outdoor environments, datasets were created from the generated worlds using the Flight Forge simulator [8]. These datasets were then used to evaluate selected mapping frameworks and to demonstrate their performance when processing large-scale outdoor environments.

4.1 Generated Datasets

Created environments from previous Sec. 3.5 were used to generate datasets containing Light Detection and Ranging (LiDAR)s, RGB cameras and ground truth segmentations. Generated datasets are listed in Table 4.1.

ID	File name	Duration	Description
0101	desert_oasis_loop_2026-05-05_20-32-59_0.mcap	13 min 7 s	Flight around lake in the oasis
0102	desert_oasis_palms_2026-05-05_22-57-19_0.mcap	12 min 25 s	Flight through palms at the oasis
0103	desert_oasis_vonder_2026-05-05_22-15-50_0.mcap	12 min 16 s	Flight in a desert
0201	etretat_cliffs_beach_2026-05-06_23-01-09_0.mcap	12 min 29 s	Flight above water on a coast
0202	etretat_cliffs_cliff_side_2026-05-07_00-26-46_0.mcap	13 min 33 s	Flight near a cliff side
0203	etretat_cliffs_road_2026-05-06_23-46-21_0.mcap	13 min 51 s	Flight along a road
0301	grand_canyon_hills_2026-05-08_21-06-16_0.mcap	14 min 16 s	Flight on hills of the canyon
0302	grand_canyon_river_2026-05-08_20-09-50_0.mcap	11 min 54 s	Flight along a river inside the canyon
0401	studlagil_canyon_hills_2026-05-08_22-46-09_0.mcap	12 min 35 s	Flight on hills of the canyon
0402	studlagil_canyon_river_2026-05-08_23-33-52_0.mcap	13 min 26 s	Flight along a river inside the canyon

Table 4.1: List of created datasets on generated worlds.

Point cloud density varies between individual datasets. During dataset creation, the simulated flights were controlled in a more realistic manner rather than following a fixed altitude or trajectory. For example, the drone was flown at higher altitudes above hilly terrain to avoid obstacles. Since the LiDAR sensor has a limited range, this resulted in sparser point

clouds. In contrast, flying close to river banks produced denser point clouds. As a result, the local point cloud density may differ from the density typically found in established training datasets.

4.2 Mapping Frameworks Verification

Three mapping frameworks that support Robot Operating System 2 (ROS2) were chosen: OctoMap [21], WaveMap [15], and volumetric/occupancy-based mapper [2]. The selected mapping frameworks differ in their primary purpose, but they partially overlap in the supported map representations. The overlap is mainly visible in semantic mapping, Euclidean Signed Distance Field (ESDF) based representations, and occupancy mapping. On Table 4.2 is shown which features are supported by which mapping frameworks.

Mapping Framework	ESDF	Occupancy mapping
Volumetric/occupancy-based mapper	Yes	Yes
OctoMap	No	Yes
WaveMap	Yes	No

Table 4.2: Table of mapping frameworks supported features.

In general, mapping frameworks are often evaluated on datasets that do not perfectly correspond to real-world conditions. For this reason, the selected mapping frameworks were evaluated using the datasets created in this thesis to demonstrate their behavior when processing large outdoor environments. The observed metrics were point cloud insertion time, memory consumption, and CPU usage. These metrics are visualized over time in Figs. 4.1–4.3 using datasets with IDs 0102, 0201, 0302 and 0402.

Fig. 4.1 shows the point cloud insertion time. WaveMap has a relatively high insertion time at the beginning of the dataset playback. After the initial phase, the insertion time stabilizes, with occasional spikes reaching approximately 40 ms. OctoMap has a high point cloud insertion time compared to the other evaluated frameworks. This may be related to the octree-based data structure used by OctoMap, which can be less efficient for this type of input than the data structures used by the other evaluated frameworks [2, p. 30].

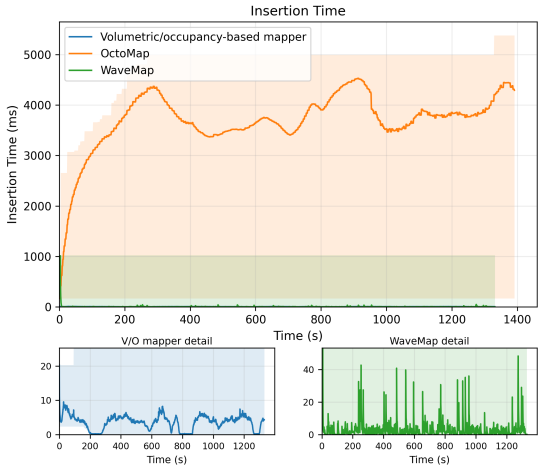
The volumetric/occupancy-based mapper performs the most consistently among the evaluated frameworks, reaching approximately 400× lower insertion times than OctoMap. This result is consistent with the expectation that the mapper should be faster than OctoMap, as reported in [2], where a speedup of 18.5× was observed. The larger difference measured in this thesis may be caused by differences in dataset loading, storage transfer speeds, or the less efficient data structure utilized by OctoMap.

Memory consumption over time is shown in Fig. 4.2. OctoMap shows a steady, approximately linear increase in memory consumption. The volumetric/occupancy-based mapper requires only a fraction of the memory used by OctoMap. This behavior is consistent with the explanation given in [2], where the lower memory consumption is attributed to the more efficient sparse voxel representation of the VDB data structure compared to the octree-based storage used by OctoMap [2, p. 64]. The same trend is also visible in our results.

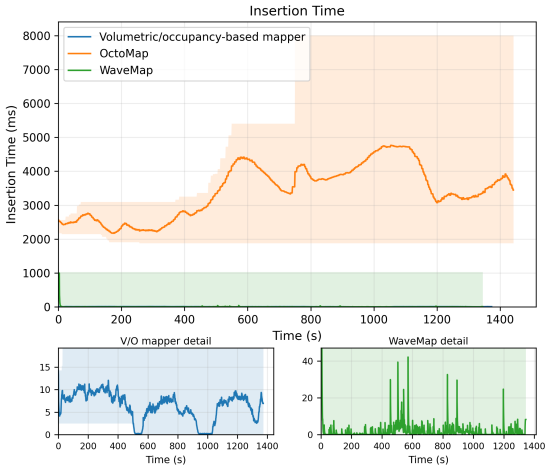
WaveMap steadily consumes approximately 45 MB of memory. One possible explanation for this low memory consumption is the different type of map representation used

by WaveMap. Unlike the other evaluated frameworks, which perform occupancy mapping, WaveMap computes an ESDF, which is in line with the claims in [15].

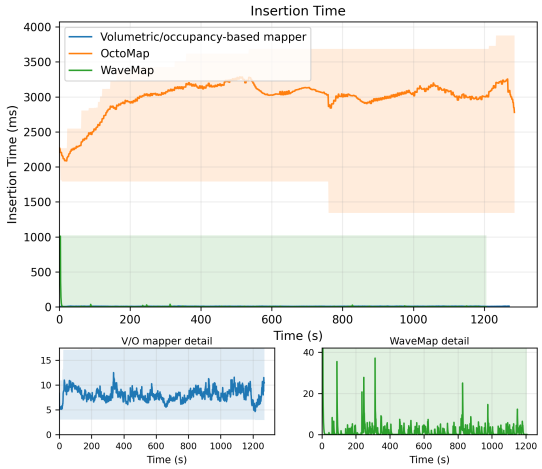
Fig. 4.3 illustrates the CPU usage of the evaluated mapping frameworks. Across the four processed datasets, all frameworks exhibit similar overall CPU usage patterns. OctoMap shows the highest CPU usage, reaching approximately 250%, which corresponds to about 2.5 CPU cores. WaveMap has the lowest CPU usage among the evaluated frameworks, remaining around 5% on average and peaking at approximately 12%. The volumetric/occupancy-based mapper shows a gradual increase in CPU usage over time, staying around 50% for most of the dataset playback and reaching peaks of approximately 100%.



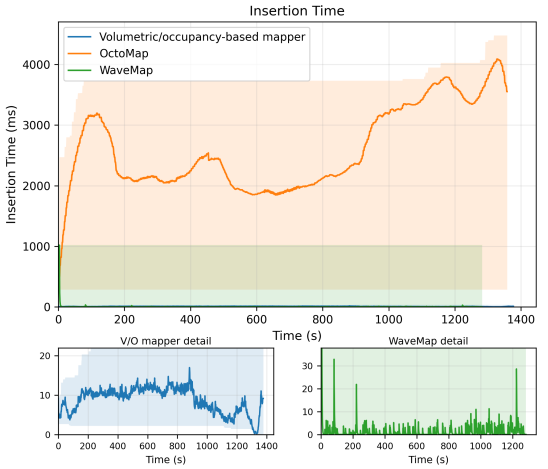
(a) Point cloud insertion time over the duration of dataset experiment 0102



(b) Point cloud insertion time over the duration of dataset experiment 0201

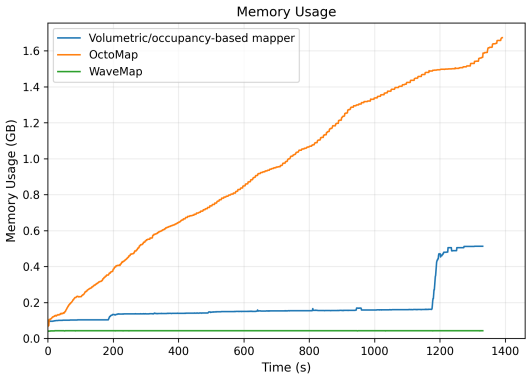


(c) Point cloud insertion time over the duration of dataset experiment 0302

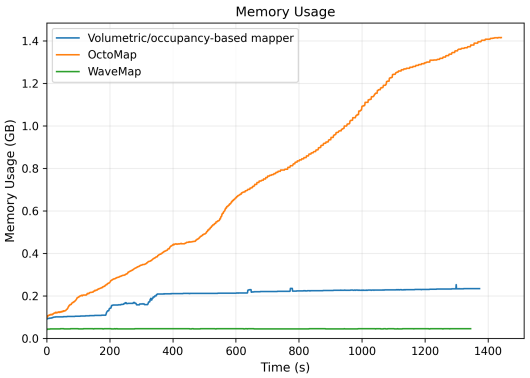


(d) Point cloud insertion time over the duration of dataset experiment 0402

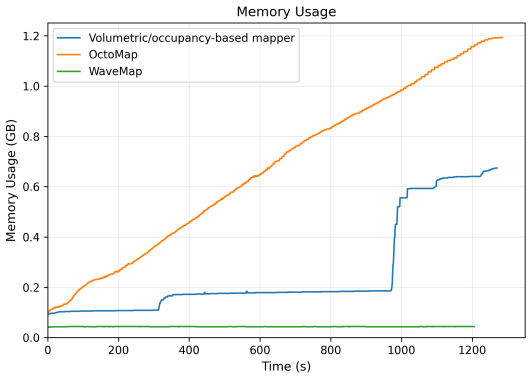
Figure 4.1: Point cloud insertion time comparison between Volumetric/occupancy-based mapper (blue), OctoMap (orange) and WaveMap (green) over the duration of each dataset experiment with min-max shaded reagions.



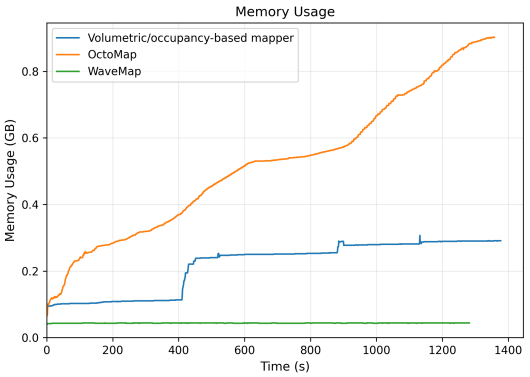
(a) Memory consumption over the duration of dataset experiment 0102



(b) Memory consumption over the duration of dataset experiment 0201

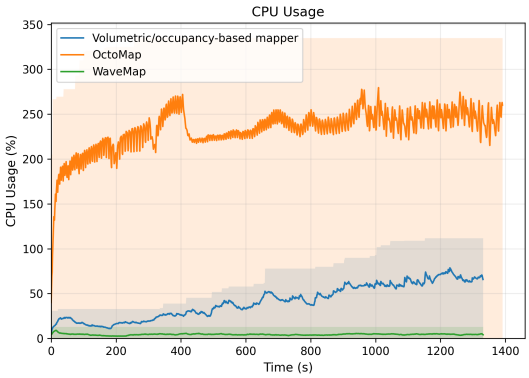


(c) Memory consumption over the duration of dataset experiment 0302

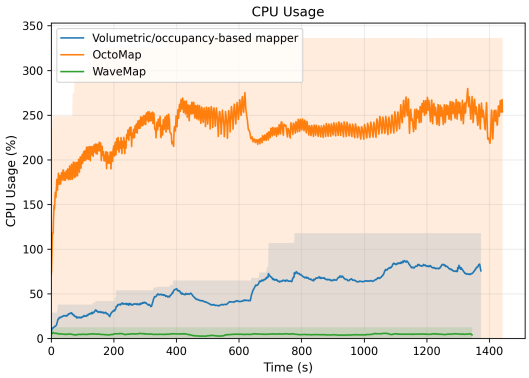


(d) Memory consumption over the duration of dataset experiment 0402

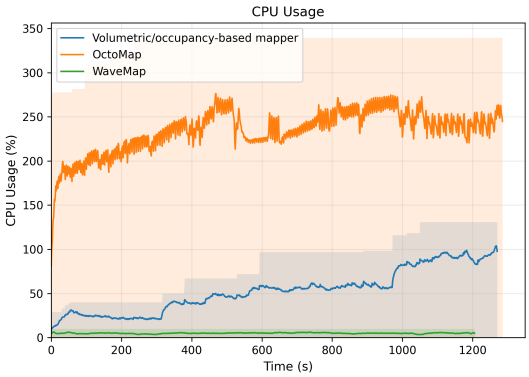
Figure 4.2: Memory consumption (RSS) comparison between Volumetric/occupancy-based mapper (blue), OctoMap (orange) and WaveMap (green) over the duration of each dataset experiment.



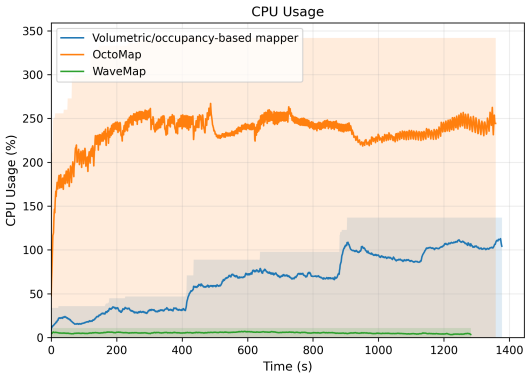
(a) CPU usage over the duration of dataset experiment 0102



(b) CPU usage over the duration of dataset experiment 0201



(c) CPU usage over the duration of dataset experiment 0302



(d) CPU usage over the duration of dataset experiment 0402

Figure 4.3: CPU usage comparison between Volumetric/occupancy-based mapper (blue), OctoMap (orange) and WaveMap (green) over the duration of each dataset experiment with min-max shaded reagions.

Chapter 5

Conclusion

This thesis introduced a possible workflow for semi-automatic generation of outdoor simulation environments in the form of an Unreal Engine 5 plugin. The resulting plugin follows common Unreal Engine 5 workflows and allows the user to adjust individual steps of the generation pipeline. The main focus of the plugin is the creation of Unreal Engine 5 world features using splines extracted from segmentation image.

Instead of creating the entire scene automatically, the plugin provides the user a tool to create the initial structure of the simulation environment. As the generated world is made entirely from existing Unreal Engine 5 features and objects, user can freely adjust and modify the generated result. This makes the tool suitable for workflows where both automation and manual control are required.

The implemented features include landscape generation from a height map, landscape splines, rivers and lakes, and biomes regions. Since the generated features are based on a common spline representation, the system can be extended with additional feature types, that follows similar workflows, in the future. In addition, biome regions generation require the user to provide Biome Templates and Biome Assets, and other objects in accordance with the PCG Biome Core workflow.

Four generated worlds were used to create datasets using Flight Forge simulator. These datasets were then used for mapping framework evaluation accordingly to the thesis assignment. Evaluated mapping frameworks were: OctoMap, WaveMap, and volumetric/occupancy-based mapper [2]. Observed metrics were: point cloud insertion time, memory consumption and CPU usage. The obtained results show how the mapping frameworks perform when processing large outdoor environments.

5.1 Future Work

Although the methods and plugin introduced in this thesis provide a reliable simulation environments creation tool, the system can be further enhanced in several ways. Possible improvements include support for additional Unreal Engine 5 object types, new generation features, and different forms of input data.

One possible extension of this plugin is to include urban generation. As discussed in related works, significant progress has been made in procedural generation of urban environments. Additionally, generation of buildings could also be achieved procedurally, easing the need of pre-created assets to generate the environments.

Another possible direction is the automatic creation of input data. In the current workflow, the height map and segmentation image have to be provided by the user. neural networks based methods, such as GAN and CGAN models, could be used to generate height map from existing satellite images. This could reduce the dependency on publicly available data, which

can be incomplete, inaccurate or imprecise. In addition, generation of completely new satellite image could make it possible to create entirely synthetic worlds, increasing the variety of generated environments.

To further expand the possibilities of automatic input data creation, LLM-guided environment generation could be explored. The user could generate input data, a basic environment layout, or even an entire environment from a text prompt. If integrated properly, this approach could significantly reduce the time required for world creation and decrease the amount of manual adjustment needed after generation. In addition, an LLM-based control layer could help identify or correct basic inconsistencies in the generated environment.

The generation of environment assets is another possible area for improvement. At the moment, assets used in the generated environments must be created manually or obtained from existing asset libraries. Procedural asset generation methods, such as those used in Infinigen, Infinigen Indoors and Infinigen-Articulated, could be adapted to generate reusable assets instead of complete photorealistic scenes. Such an extension could reduce the need for manually prepared assets and make it easier to generate unique environments.

Chapter 6

References

[1] Y. Zhuang et al., *SimWorld-Robotics: Synthesizing Photorealistic and Dynamic Urban Environments for Multimodal Robot Navigation and Collaboration*, arXiv:2512.10046 [cs], Jan. 2026. DOI: 10.48550/arXiv.2512.10046 Accessed: May 9, 2026. [Online]. Available: http://arxiv.org/abs/2512.10046

[2] D. Čapek, "Volumetric Mapping and Planning Framework for Autonomous Long-Range Navigation of UAVs in GNSS Denied Environments," M.S. thesis, Czech Technical University in Prague, Faculty of Electrical Engineering, 2026. Accessed: May 19, 2026. [Online]. Available: https://hdl.handle.net/10467/178701

[3] H. W. G. B, A. Dewandaru, and T. B. Wicaksono, "Procedural Content Generation with Large Language Models for Three-Dimensional Environment Design," in *2025 IEEE International Conference on Data and Software Engineering (ICoDSE)*, ISSN: 2640-0227, Oct. 2025, pp. 413–418. DOI: 10.1109/ICoDSE68111.2025.11351639 Accessed: May 10, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11351639

[4] J. Johansson and A. Riis, *Satellite-driven environment reconstruction in Unreal Engine : A simplified process for widespread adoption*, eng. 2025. Accessed: May 9, 2026. [Online]. Available: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-28349

[5] A. Joshi et al., *Procedural Generation of Articulated Simulation-Ready Assets*, _eprint: 2505.10755, 2025. [Online]. Available: https://arxiv.org/abs/2505.10755

[6] J. McMillen and Y. Yilmaz, "SegGen: An Unreal Engine 5 Pipeline for Generating Multimodal Semantic Segmentation Datasets," en, *Sensors*, vol. 25, no. 17, p. 5569, Jan. 2025, ISSN: 1424-8220. DOI: 10.3390/s25175569 Accessed: May 9, 2026. [Online]. Available: https://www.mdpi.com/1424-8220/25/17/5569

[7] F.-Y. Sun et al., *3D-Generalist: Self-Improving Vision-Language-Action Models for Crafting 3D Worlds*, arXiv:2507.06484 [cs], Aug. 2025. DOI: 10.48550/arXiv.2507.06484 Accessed: May 9, 2026. [Online]. Available: http://arxiv.org/abs/2507.06484

[8] D. Čapek et al., *FlightForge: Advancing UAV Research with Procedural Generation of High-Fidelity Simulation and Integrated Autonomy*, Publication Title: arXiv preprint arXiv: 2502.05038, 2025. [Online]. Available: https://arxiv.org/abs/2502.05038v1

[9] K. Ebadi et al., "Present and Future of SLAM in Extreme Environments: The DARPA SubT Challenge," *IEEE Transactions on Robotics*, vol. 40, pp. 936–959, 2024, ISSN: 1941-0468. DOI: 10.1109/TRO.2023.3323938 Accessed: May 9, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10286080

[10] J.-H. Liu, S.-K. Zhang, C. Zhang, and S.-H. Zhang, "Controllable Procedural Generation of Landscapes," in *Proceedings of the 32nd ACM International Conference on Multimedia*, ser. MM '24, New York, NY, USA: Association for Computing Machinery, Oct. 2024, pp. 6394–6403, ISBN: 979-8-4007-0686-8. DOI: 10.1145/3664647.3681129 Accessed: May 10, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3664647.3681129

[11] A. Raistrick et al., "Infinigen Indoors: Photorealistic Indoor Scenes using Procedural Generation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2024, pp. 21 783–21 794.

[12] S. Zhang et al., *CityX: Controllable Procedural Content Generation for Unbounded 3D Cities*, arXiv:2407.17572 [cs.CV], Dec. 2024. DOI: 10.48550/arXiv.2407.17572 Accessed: May 22, 2026. [Online]. Available: http://arxiv.org/abs/2407.17572

[13] Z. Chen, G. Wang, and Z. Liu, "SceneDreamer: Unbounded 3D Scene Generation from 2D Image Collections," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 12, pp. 15 562–15 576, Dec. 2023, arXiv:2302.01330 [cs.CV], ISSN: 0162-8828, 2160-9292, 1939-3539. DOI: 10.1109/TPAMI.2023.3321857 Accessed: May 22, 2026. [Online]. Available: http://arxiv.org/abs/2302.01330

[14] A. Raistrick et al., "Infinite Photorealistic Worlds Using Procedural Generation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 12 630–12 641.

[15] V. Reijgwart, C. Cadena, R. Siegwart, and L. Ott, "Efficient Volumetric Mapping of Multi-Scale Environments Using Wavelet-Based Compression," in *Robotics: Science and Systems XIX*, 2023. DOI: 10.15607/RSS.2023.XIX.065

[16] J. Freiknecht and W. Effelsberg, "Procedural Generation of Multistory Buildings With Interior," *IEEE Transactions on Games*, vol. 12, no. 3, pp. 323–336, Sep. 2020, ISSN: 2475-1510. DOI: 10.1109/TG.2019.2957733 Accessed: May 10, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8926482

[17] E. Panagiotou and E. Charou, *Procedural 3D Terrain Generation using Generative Adversarial Networks*, arXiv:2010.06411 [eess], Oct. 2020. DOI: 10.48550/arXiv.2010.06411 Accessed: May 10, 2026. [Online]. Available: http://arxiv.org/abs/2010.06411

[18] H. Jain and A. P. Kumar, *A Sequential Thinning Algorithm For Multi-Dimensional Binary Patterns*, en, Oct. 2017. Accessed: Mar. 29, 2026. [Online]. Available: https://arxiv.org/abs/1710.03025v2

[19] A. Tsirikoglou, J. Kronander, M. Wrenninge, and J. Unger, *Procedural Modeling and Physically Based Rendering for Synthetic Data Generation in Automotive Applications*, en, Oct. 2017. Accessed: May 9, 2026. [Online]. Available: https://arxiv.org/abs/1710.06270v2

[20] N. Mikuličić and Mihajlović, "Procedural generation of mediterranean environments," in *2016 39th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, May 2016, pp. 261–266. DOI: 10.1109/MIPRO.2016.7522149 Accessed: May 10, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7522149

[21] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," *Autonomous Robots*, vol. 34, pp. 189–206, 2013. DOI: 10.1007/s10514-012-9321-0

[22] P. F. Felzenszwalb and D. P. Huttenlocher, "Distance Transforms of Sampled Functions," *Theory of Computing*, vol. 8, pp. 415–428, Sep. 2012, Number: 19. DOI: 10.4086/toc.2012.v008a019 Accessed: Mar. 29, 2026. [Online]. Available: https://theoryofcomputing.org/articles/v008a019/

[23] B. Bradley, "Towards the procedural generation of urban building interiors," *The University of Hull*, 2005.

[24] Z. Guo and R. W. Hall, "Fast fully parallel thinning algorithms," *CVGIP: Image Understanding*, vol. 55, no. 3, pp. 317–328, May 1992, ISSN: 1049-9660. DOI: 10.1016/1049-9660(92)90029-3 Accessed: Mar. 29, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/1049966092900293

[25] E. T. Y. Lee, "Choosing nodes in parametric curve interpolation," *Computer-Aided Design*, vol. 21, no. 6, pp. 363–370, Jul. 1989, ISSN: 0010-4485. DOI: 10.1016/0010-4485(89)90003-1 Accessed: Apr. 9, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0010448589900031

[26] D. H. Douglas and T. K. Peucker, *ALGORITHMS FOR THE REDUCTION OF THE NUMBER OF POINTS REQUIRED TO REPRESENT A DIGITIZED LINE OR ITS CARICATURE | Cartographica*. Accessed: Mar. 29, 2026. [Online]. Available: https://utppublishing.com/doi/abs/10.3138/fm57-6770-u75u-7727

[27] Epic Games, *Procedural Content Generation Biome Core Overview*, en-US. Accessed: May 2, 2026. [Online]. Available: https://dev.epicgames.com/documentation/unreal-engine/procedural-content-generation-pcg-biome-core-and-sample-plugins-overview-guide-in-unreal-engine

[28] Epic Games, *Procedural Content Generation Overview*, en-US. Accessed: May 2, 2026. [Online]. Available: https://dev.epicgames.com/documentation/unreal-engine/procedural-content-generation-overview

[29] C. Yuksel, S. Schaefer, and J. Keyser, *On the parameterization of Catmull-Rom curves | 2009 SIAM/ACM Joint Conference on Geometric and Physical Modeling*. Accessed: Mar. 29, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/1629255.1629262

[30] T. Y. Zhang and C. Y. Suen, *A fast parallel algorithm for thinning digital patterns | Communications of the ACM*. Accessed: Mar. 29, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/357994.358023